UNIVERSIDAD MAYOR

FACULTAD DE CIENCIAS, INGENIERÍA Y TECNOLOGÍA

Escuela de Ingeniería Civil en Computación e Informática

Asignatura: Arquitectura de Desarrollo (Móvil y Web) · Código: INGT1003-001

PROYECTO DRIWIN

Hito 2: El Motor que Sostiene la Promesa

Documento Técnico de Arquitectura Back-End, API REST y Persistencia

---

Nombre del Equipo:

Equipo DriWin

Integrantes del Proyecto (en orden alfabético por apellido):

Acevedo, Vicente (21.626.029-5) · GitHub: @VinJimbo97

Cáceres, Javier (21.911.171-1) · GitHub: @Ash-Lee213

Castillo, Joaquín (21.723.477-8) · GitHub: @joaquincastilloc

Docente:

Prof. Carlos Muñoz S.

Repositorio Oficial:

https://github.com/VinJimbo97/DriWinProyect.git

Santiago, Chile · 30 de octubre de 2026

Índice General

Sección del Documento	Pág. Ref.
Introducción	3
1. Trazabilidad y Evolución Arquitectónica respecto del Hito 1 (Criterio B1)	4
1.1. Línea Base Propuesta en el Hito 1	4
1.2. Bitácora de Cambios, Refinamiento Dual y Justificación de Impacto	4
1.3. Cronología Verificada en el Historial de Git	6
2. Justificación del Stack Tecnológico y Dependencias del Servidor (Criterio A2)	7
2.1. Enfoque Dual de la Capa de Negocio: Rust Local vs. Node.js/TypeScript Cloud	7
2.2. Matriz Comparativa de Tecnologías de Servidor y Costos Asumidos	8
2.3. Inventario de Dependencias Verificadas y Rol Técnico en DriWin	9
3. Decisión Arquitectónica: Monolito Modular vs. Distribuido (Criterio A3)	11
3.1. Postura Adoptada: Monolito de Metadatos con Offloading Binario	11
3.2. Defensa por Carga Esperada, Tamaño del Equipo y Costo de Operación	11
3.3. Complejidad de Despliegue y Umbrales Objetivos de Revisión	13
4. Diseño y Contrato de la Interfaz de Programación REST (Criterios B2 y A1)	14
4.1. Criterio de Versionado, Cabeceras y Envoltorios Estándar de Datos y Error	14
4.2. Mapeo de Errores SQLSTATE de PostgreSQL hacia Códigos de Estado HTTP	15
4.3. Tabla Maestra de Recursos y Operaciones CRUD	16
4.4. Especificación Detallada de Contratos por Recurso y Consultas SQL	17
5. Separación de Responsabilidades y Arquitectura en Capas (Criterio B3)	22
5.1. Árbol de Directorios del Proyecto y Clasificación en Cinco Capas	22
5.2. Regla Unidireccional de Dependencia y Aislamiento de Responsabilidades	23
5.3. Recorrido Completo de una Petición de Punta a Punta (Trazabilidad)	24
6. Persistencia en PostgreSQL, Modelo de Datos y Seguridad (Criterio B4)	25
6.1. Diseño del Script Reproducible e Idempotente (db/init.sql)	25
6.2. Diccionario de Datos, Claves, Restricciones e Índices de las 8 Tablas	26
6.3. Manejo de Credenciales, Variables de Entorno (.env.example) y Seguridad	28
7. Instrucciones de Ejecución Reproducible y Demostración en Vivo	29
Conclusión	30
Bibliografía (Criterio B5)	31
Anexo 1: Declaración de Uso de Inteligencia Artificial (Anexo C)	32
Anexo 2: Análisis de Protección de Datos Personales — Ley N.º 21.719 (Anexo D)	33

Sección del Documento	Pág. Ref.
Anexo 3: Matriz de Deuda de Integración UI-BD y Checklist de Cierre	34

Índice de Tablas y Figuras

Identificador	Título y Descripción
Tabla 1	Bitácora de trazabilidad y evolución de artefactos entre el Hito 1 y el Hito 2.
Tabla 2	Cronología verificada de confirmaciones (commits) en el repositorio Git.
Tabla 3	Matriz comparativa para la elección del runtime y framework de la API REST.
Tabla 4	Matriz comparativa para la elección del motor de base de datos relacional en la nube.
Tabla 5	Stack de paquetes verificados para la API REST (Node.js + Express + TypeScript).
Tabla 6	Stack verificado de la Capa de Presentación y de la Capa de Negocio Local (Tauri/Rust).
Tabla 7	Clasificación arquitectónica de componentes: Monolito modular vs. Servicios externos.
Tabla 8	Análisis de volumen de tráfico de red y efecto del patrón de Offloading binario.
Tabla 9	Umbrales cuantitativos para la revisión de la postura monolítica hacia microservicios.
Tabla 10	Catálogo de códigos de estado HTTP y mapeo técnico desde SQLSTATE (PostgreSQL 16).
Tabla 11	Tabla maestra de recursos expuestos por la API REST (/api/v1) y cobertura CRUD.
Tabla 12	Contrato de endpoints para el Recurso 1: hardware-components.
Tabla 13	Contrato de endpoints para el Recurso 2: driver-packages.
Tabla 14	Contrato de endpoints para los Recursos 3, 4 y 5 (apps, bloatware y perfiles).
Tabla 15	Clasificación de archivos del repositorio según las 5 capas de la arquitectura.
Tabla 16	Trazabilidad paso a paso del recorrido de una petición desde la ruta hasta la base de datos.
Tabla 17	Diccionario relacional consolidado de las 8 tablas del esquema driwin_catalog.
Tabla 18	Auditoría de seguridad, barrido de secretos en Git y matriz de controles técnicos.
Figura 1	Diagrama de trazabilidad funcional de las capas entre el Hito 1 y el Hito 2.
Figura 2	Árbol completo de directorios y separación modular en capas del proyecto DriWin.
Figura 3	Diagrama Entidad-Relación (DER) textual de las 8 tablas en PostgreSQL 16.

Introducción

La configuración post-instalación y el mantenimiento continuo de equipos con sistema operativo Windows representan un cuello de botella operativo tanto para usuarios finales de alta exigencia como para técnicos de soporte y micro, pequeñas y medianas empresas (MIPYMES). Según se documentó en la propuesta comercial del Hito 1 (Acevedo et al., 2026), la proliferación de software preinstalado innecesario (bloatware) y la asignación de controladores genéricos degradan el rendimiento del hardware, induciendo una obsolescencia prematura que contribuye a las más de 220.000 toneladas anuales de residuos de aparatos eléctricos y electrónicos (RAEE) en Chile (Ministerio del Medio Ambiente, 2026) y exponiendo a las organizaciones a incumplimientos frente a la Ley Marco de Ciberseguridad N.º 21.663 y la Ley de Protección de Datos Personales N.º 21.719.

Para resolver esta problemática de raíz, el proyecto DriWin reemplaza el paradigma tradicional de los actualizadores de controladores manuales por una arquitectura basada en intenciones de uso. Al iniciar el sistema, el usuario define el propósito productivo de su máquina mediante cuatro perfiles canónicos (Gamer, Creator, Developer y Office). En paralelo, el cliente inspecciona los identificadores físicos de fabricante y dispositivo (VEN_ID y DEV_ID) a través de las interfaces de administración de Windows, consultando a un servicio centralizado en la nube que determina con precisión matemática qué edición específica de controlador (por ejemplo, priorizando baja latencia Adrenalin/Game Ready frente a estabilidad Studio o certificación WHQL minimalista), qué lote de aplicaciones recomendadas y qué reglas de purga de bloatware corresponden a esa combinación única de hardware e intención.

El presente Documento Técnico corresponde a la entrega formal del Hito 2 de la asignatura Arquitectura de Desarrollo (Móvil y Web) [INGT1003] y tiene como objetivo especificar, fundamentar y documentar la construcción del motor del lado del servidor (Back-End) que sostiene la promesa comercial formulada en septiembre. Siguiendo los lineamientos metodológicos de ingeniería de software (Pressman, 2009; Vara Mesa et al., 2015), este informe transparenta el estado exacto del repositorio auditado en la rama feature/auditoria-vibecode (commit 01375c0), distinguiendo rigurosamente entre los componentes ya implementados y verificados en código (interfaz cliente en React 19.2.8, modelo relacional reproducible de 8 tablas en PostgreSQL 16, almacenamiento de objetos en Azure Blob Storage y políticas de resguardo de credenciales) y el contrato de programación de aplicaciones (API REST) estructurado en capas para su ejecución en vivo.

A lo largo del documento, el lector encontrará siete capítulos técnicos articulados directamente con la rúbrica de evaluación de la Unidad 2: (1) la trazabilidad detallada y la bitácora de cambios respecto del Hito 1; (2) la justificación contrastada del stack tecnológico y los costos asumidos por cada decisión; (3) la defensa arquitectónica de nuestra postura de monolito modular de metadatos con descarga delegada (offloading binario); (4) el contrato completo de la API REST (/api/v1) con operaciones CRUD, cargas útiles JSON y mapeo de errores SQLSTATE; (5) la separación estricta en cinco capas y el recorrido de punta a punta de una petición; (6) el diseño de persistencia en Azure Database for PostgreSQL Flexible Server junto con el manejo seguro de variables de entorno; y (7) el manual de ejecución reproducible para levantar y verificar el servicio en un equipo limpio.

1. Trazabilidad y Evolución Arquitectónica respecto del Hito 1

1.1. Línea Base Propuesta en el Hito 1

En el documento de propuesta entregado en el Hito 1 (Propuesta_Tecnica.pdf, secciones 7 y 8), el equipo definió los cimientos conceptuales de DriWin bajo un modelo cliente-servidor organizado en tres macro-capas: (a) una Capa de Presentación construida con TypeScript, React y CSS; (b) una Capa de Negocio dividida en dos ámbitos de actuación —un ámbito local encargado de ejecutar comandos de PowerShell/WMI en segundo plano para extraer identificadores VEN/DEV y desinstalar bloatware, y un ámbito de servidor consistente en una API alojada en Microsoft Azure encargada de cruzar dichos identificadores con el perfil del usuario—; y (c) una Capa de Datos en la nube de Azure destinada a almacenar el catálogo de componentes PCI/USB ID y sus respectivas rutas de descarga.

En la evaluación comparativa de lenguajes del Hito 1 se seleccionó la combinación de TypeScript (React) y Rust por sobre las alternativas de C++ con Qt y C# con WinUI/MVVM. El argumento central fue que el integrante Vicente Acevedo ya poseía dominio previo de TypeScript, lo que aceleraba la construcción visual, mientras que Rust aportaba seguridad de memoria y acceso nativo a bajo nivel para interactuar con el sistema operativo. Sin embargo, el documento del Hito 1 dejó abierta la especificación del framework concreto para la API REST en la nube, el motor específico de base de datos relacional en Azure y el mecanismo físico de distribución de los instaladores binarios.

1.2. Bitácora de Cambios, Refinamiento Dual y Justificación de Impacto

Durante la transición hacia el Hito 2, el equipo sometió la propuesta inicial a pruebas de viabilidad técnica, costos operativos en Azure for Students y tiempos reales de desarrollo. Como resultado de este análisis de ingeniería, se ejecutaron seis refinamientos arquitectónicos debidamente documentados y controlados en el repositorio. La Tabla 1 presenta la bitácora oficial de cambios, detallando qué se movió respecto del Hito 1, el motivo técnico y de gestión que originó cada cambio y el impacto concreto sobre el sistema.

Tabla 1. Bitácora de trazabilidad y evolución de artefactos entre el Hito 1 y el Hito 2.

Componente / Artefacto	Estado en Hito 1 (Septiembre 2026)	Estado en Hito 2 (Octubre 2026)	Por qué se movió (Motivo Técnico y de Gestión)	Impacto Arquitectónico y en el Proyecto
1. Capa de Negocio (Servidor vs. Local)	Rust declarado de forma general para todo el Back-End sin distinguir entre el binario local y la API web en Azure.	Enfoque Dual: Rust (Tauri 2.12.4) confinado a la Capa Local; Node.js + Express 5.3.0 (TypeScript 6.0.3) como runtime de la API REST Cloud.	La API en la nube no interactúa con hardware físico; su carga es 100% I/O-bound (consultas SQL y despacho de JSON). Usar TypeScript en el servidor unifica el lenguaje y las interfaces DTO con el Front-End (React), reduce la curva de	Separa limpiamente src-tauri/ (acceso privilegiado WMI/pnputil en el PC cliente) de server-api/ (API REST limpia en 3 capas internas: routes, services y repositories), eliminando el acoplamiento entre el sistema operativo local

Componente / Artefacto	Estado en Hito 1 (Septiembre 2026)	Estado en Hito 2 (Octubre 2026)	Por qué se movió (Motivo Técnico y de Gestión)	Impacto Arquitectónico y en el Proyecto
			aprendizaje para los 3 integrantes y garantiza el cumplimiento de plazos.	y la nube.
2. Motor de Base de Datos en Azure	Mencionado genéricamente como 'Base de datos en Azure' sin motor ni esquema relacional definido.	Azure Database for PostgreSQL Flexible Server 16 (instancia Burstable B1ms) con esquema driwin_catalog.	Se contrastó Azure SQL Database frente a PostgreSQL Flexible Server. Se eligió PostgreSQL 16 por su soporte nativo de arreglos (TEXT[]) con índices GIN, restricciones CHECK avanzadas, cumplimiento del estándar SQL y menor costo base en créditos estudiantiles.	Creación del script reproducible e idempotente db/init.sql (449 líneas) con integridad referencial estricta (ON DELETE RESTRICT / CASCADE) y 9 consultas de validación.
3. Alcance del Modelo de Datos	Conceptual: solo se contemplaba relacionar ID de hardware con nombre comercial y enlace de driver.	Modelo relacional expandido a 8 tablas normalizadas, 4 índices, 5 claves foráneas y 15 restricciones con nombre.	Para soportar el caso de uso completo del Hito 1 (drivers, catálogo de aplicaciones por perfil, limpieza de bloatware y planes comerciales Premium) era indispensable modelar todas las entidades en la base de datos.	Expansión verificada en Git desde 3 tablas iniciales (commit e857fd0) a 8 tablas completas (commit 01375c0), habilitando 5 recursos con soporte CRUD completo.
4. Distribución de Binarios (.exe / .msi)	Se planteaba redirigir al usuario a 'URL de descarga directa de los fabricantes' externos.	Almacenamiento propio en Azure Blob Storage (cuenta driwinstorage, contenedores apps y drivers, nivel Hot LRS) bajo patrón de Offloading.	Los enlaces externos de fabricantes cambian sin previo aviso, usan redirecciones dinámicas y rompen los parámetros de instalación silenciosa (/S, /qn). Alojar versiones validadas en nuestro Blob Storage garantiza determinismo en la demostración.	Se poblaron y mapearon 20 instaladores en el contenedor apps/ (~6,03 GB) y 12 paquetes en drivers/ (~4,31 GB). La API REST solo transfiere metadatos JSON (~1 KB), delegando los ~10,34 GB de tráfico pesado a Azure Blob Storage.
5. Estrategia de Recolección de Catálogo	Supuesto de consulta directa a repositorios universales PCI/USB ID en tiempo real.	Estrategia en 2 fases (EstrategiaRecoleccionDatos.md): Fase 1 Semilla Manual Controlada (activa en Hito 2) y Fase 2 Ingesta Automatizada (futura).	Consultar APIs de terceros o ejecutar web scraping en vivo durante la evaluación introduce puntos únicos de falla por bloqueos de red o	Aísla la demostración del Hito 2 de fallas externas mediante un catálogo semilla determinista en db/init.sql y documenta los

Componente / Artefacto	Estado en Hito 1 (Septiembre 2026)	Estado en Hito 2 (Octubre 2026)	Por qué se movió (Motivo Técnico y de Gestión)	Impacto Arquitectónico y en el Proyecto
			cambios de DOM.	requisitos legales para una futura Fase 2.
6. Gobernanza Legal y Seguridad de Secretos	No existían archivos .gitignore, licencias de software ni auditoría de dependencias en el Hito 1.	Incorporación de .gitignore multicapa, LICENSE (MIT), THIRD_PARTY_LICENSES.md y auditoría de 70 paquetes npm.	Cumplir con el estándar profesional del Anexo B (cero credenciales en el repositorio) y verificar que ninguna dependencia introduzca licencias copyleft incompatibles (GPL/AGPL).	Historial de Git 100% libre de credenciales (0 filtraciones en 41 commits) y trazabilidad legal completa de las 70 dependencias resueltas en el árbol.

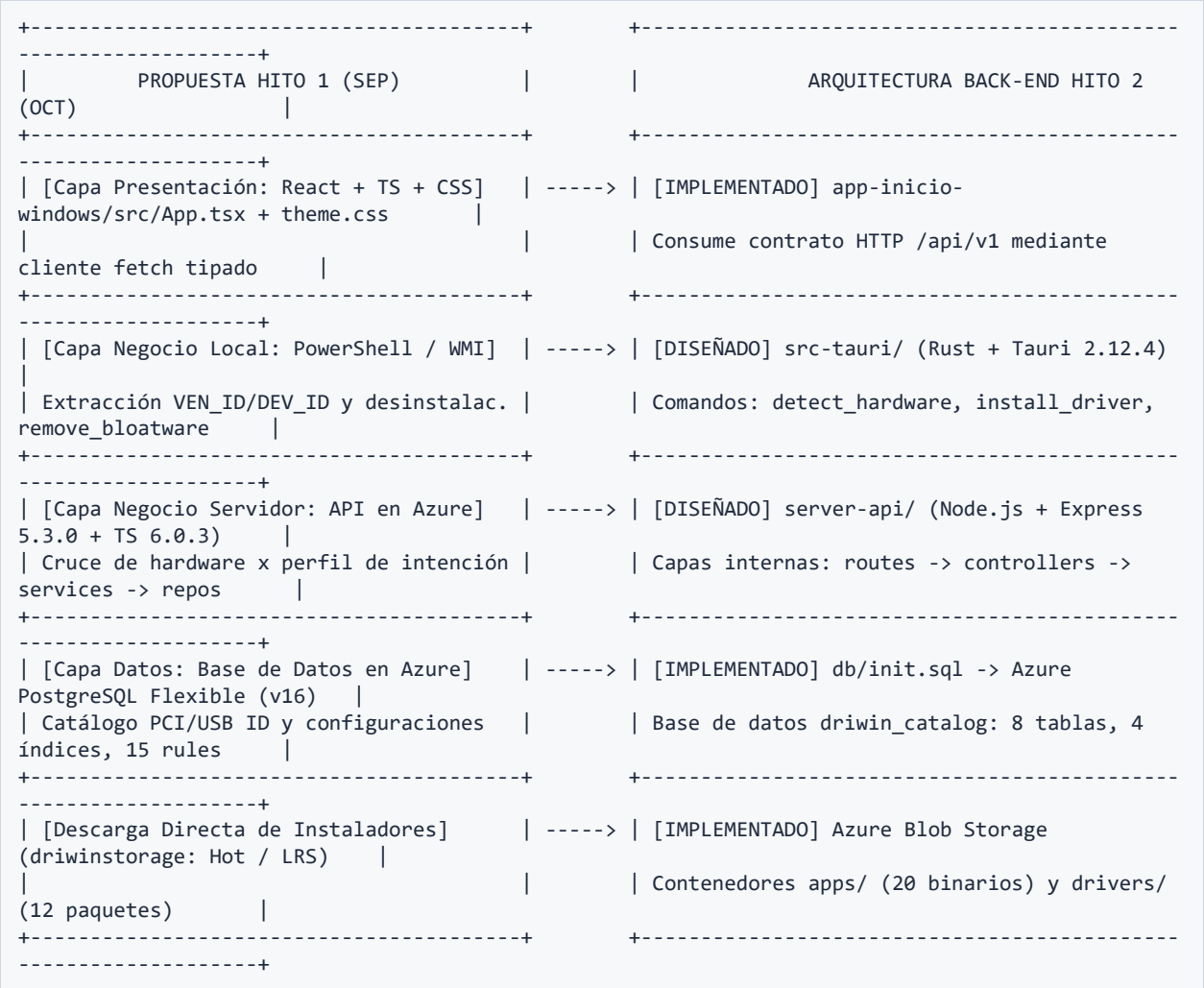
1.3. Cronología Verificada en el Historial de Git

Toda evolución arquitectónica declarada en este documento posee respaldo verificable en el historial de confirmaciones (commits) del repositorio oficial. La Tabla 2 resume los hitos de versión auditados hasta el 21 de octubre de 2026, mientras que la Figura 1 esquematiza cómo cada componente prometido en el Hito 1 se conecta con los artefactos reales del Hito 2.

Tabla 2. Cronología verificada de confirmaciones (commits) en el repositorio Git.

Hash Commit	Fecha	Autor / Rama	Mensaje de Confirmación	Impacto Técnico Verificado
47f2b7e .. 999879e	Sep - Oct 2026	Equipo DriWin (main)	Proyecto DriWin -> Propuesta de back end	Construcción del prototipo React 19 + Vite 8, estilos CSS, diagramas de flujo y PDF del Hito 1.
e857fd0	16-10-2026	Joaquín Castillo (feature/auditoria-vibecode)	Se agrega script reproducible de base de datos y catalogo semilla	Creación de la Capa de Persistencia inicial: db/init.sql con las 3 tablas nucleares y Borrador.md con justificación de PostgreSQL.
8f8b22c	17-10-2026	Joaquín Castillo (feature/auditoria-vibecode)	Integracion de licencias utilizadas	Incorporación de LICENSE (MIT), THIRD_PARTY_LICENSES.md (316 líneas) y auditoría normativa de las 70 dependencias.
e71d85d	17-10-2026	Joaquín Castillo (feature/auditoria-vibecode)	Correccion de error de seguridad	Implementación de .gitignore raíz con bloqueo estricto de archivos .env, certificados (.pem, .key, .pfx) y artefactos.
01375c0	18-10-2026	Joaquín Castillo (feature/auditoria-vibecode)	Esquema con apps por perfil, reglas bloatware y usuarios sinteticos	Expansión del modelo relacional de 3 a 8 tablas en db/init.sql (449 líneas), índices GIN/B-Tree y 9 consultas SQL de negocio.

Figura 1. Diagrama de trazabilidad funcional de las capas entre el Hito 1 y el Hito 2.



2. Justificación del Stack Tecnológico y Dependencias del Servidor

2.1. Enfoque Dual de la Capa de Negocio: Rust Local vs. Node.js/TypeScript Cloud

En la ingeniería de sistemas modernos, seleccionar una única herramienta para resolver problemas de naturaleza opuesta conduce a sobreingeniería o a cuellos de botella de rendimiento (Vara Mesa et al., 2015). En DriWin, la Capa de Negocio opera en dos entornos físicamente separados con exigencias técnicas antagónicas:

- **1. Entorno Local en el PC del Usuario (CPU-Bound y Privilegiado):** Requiere elevar permisos de administrador (UAC), invocar APIs nativas del sistema operativo Windows (WMI/CIM y registro HKLM\SYSTEM\CurrentControlSet\Enum\PCI), parsear cadenas crudas de hardware y ejecutar procesos desatendidos como pnputil.exe o msiexec.exe. Para este dominio se mantiene Rust integrado mediante el framework Tauri 2.12.4 (Tauri Contributors, 2026). Rust garantiza seguridad de memoria en tiempo de compilación sin recolector de basura y genera un binario ligero que utiliza el motor nativo WebView2 de Windows en lugar de empaquetar una instancia completa de Chromium como haría Electron.
- **2. Entorno de Servidor en la Nube (I/O-Bound y Orientado a Contratos JSON):** La API REST centralizada en Microsoft Azure jamás accede al hardware físico de ningún dispositivo. Su única responsabilidad es recibir peticiones HTTP con cargas útiles JSON ligeras (~1 KB), validar su estructura, ejecutar consultas SQL parametrizadas sobre PostgreSQL y retornar metadatos junto con la URL de descarga alojada en Azure Blob Storage. Dado que el 99% del tiempo de respuesta de este servicio corresponde a espera de entrada/salida de red (I/O de base de datos), el modelo de bucle de eventos asíncrono no bloqueante (Event Loop) de Node.js (OpenJS Foundation, 2026) con Express 5.3.0 y TypeScript 6.0.3 resulta técnicamente idóneo.

2.2. Matriz Comparativa de Tecnologías de Servidor y Costos Asumidos

Siguiendo el criterio A2 de la rúbrica, toda decisión arquitectónica debe contrastarse contra alternativas reales bajo criterios explícitos de ingeniería y declarar el costo o desventaja que el equipo acepta asumir. Las Tablas 3 y 4 documentan las matrices de decisión para el runtime/framework de la API REST y para el motor de persistencia.

Tabla 3. Matriz comparativa para la elección del runtime y framework de la API REST.

Criterio de Evaluación	Alternativa A (ELEGIDA): Node.js + Express 5.3 + TypeScript 6.0	Alternativa B (Descartada para Cloud): Rust + Axum / Actix-Web	Alternativa C (Descartada): Python + FastAPI
1. Naturaleza de la Carga (I/O vs CPU)	Óptima para I/O-bound: el Event Loop maneja cientos de conexiones concurrentes hacia PostgreSQL con mínima sobrecarga de serialización JSON nativa.	Sobredimensionada para I/O simple: su ventaja competitiva brilla en cómputo intensivo de CPU, el cual no ocurre en una API de catálogo de metadatos.	Buena mediante asyncio/uvicorn, aunque con menor rendimiento bruto en serialización concurrente frente al motor V8 de Node.js.
2. Unificación del	Total: comparte el mismo	Fragmentada: obliga a	Fragmentada: requiere

Criterio de Evaluación	Alternativa A (ELEGIDA): Node.js + Express 5.3 + TypeScript 6.0	Alternativa B (Descartada para Cloud): Rust + Axum / Actix-Web	Alternativa C (Descartada): Python + FastAPI
Stack y Tipado	compilador TypeScript 6.0.3 y las mismas interfaces de dominio (DriverPackage, RecommendedApp) con el Front-End en React.	mantener structs en Rust (Serde) duplicados respecto a las interfaces TypeScript del cliente React.	mantener modelos Pydantic en Python separados de las interfaces TypeScript del Front-End.
3. Productividad y Curva del Equipo (3 integrantes)	Alta velocidad de entrega: el equipo ya domina TypeScript desde el Hito 1, permitiendo construir y defender las 3 capas del servidor dentro del plazo.	Riesgo crítico de plazo: lidiar con lifetimes, borrow checker y traits asíncronos para un CRUD SQL retrasaría la integración del Hito 2 y 3.	Media: requeriría introducir un tercer lenguaje de programación al repositorio (TS + Rust + Python), aumentando la dispersión cognitiva.
4. Costo / Desventaja Asumida por el Equipo	COSTO ASUMIDO: Mayor consumo base de memoria RAM (~60-90 MB en reposo) y latencia de arranque en frío ligeramente superior frente a un binario nativo en Rust. MITIGACIÓN: Gracias al patrón de Offloading hacia Azure Blob Storage, el proceso Node.js nunca carga instaladores en memoria; solo despacha JSON de ~1 KB, manteniendo el consumo de RAM plano.	Costo de oportunidad en horas-hombre de desarrollo y alta complejidad de mantenimiento para un equipo de 3 estudiantes.	Costo de mantener múltiples entornos virtuales (venv/pip + npm + cargo) y tipado dinámico en tiempo de ejecución.

Tabla 4. Matriz comparativa para la elección del motor de base de datos relacional en la nube.

Criterio de Evaluación	Alternativa A (ELEGIDA): Azure Database for PostgreSQL Flexible Server 16	Alternativa B (Descartada): Azure SQL Database (MS SQL Server)	Alternativa C (Descartada): MongoDB Atlas / Azure Cosmos DB (NoSQL)
1. Integridad Relacional del Catálogo	Excelente: soporta claves compuestas, restricciones CHECK expresivas y claves foráneas con reglas diferenciadas (ON DELETE RESTRICT para perfiles y CASCADE para componentes).	Excelente: motor relacional maduro con soporte transaccional ACID completo.	Deficiente para este dominio: el cruce estricto entre hardware_components, user_profiles y driver_packages exigiría desnormalización o costosos $lookup sin integridad referencial nativa.
2. Tipos de Datos Avanzados e Indexación	Superior: soporte nativo de arreglos tipados (TEXT[]) indexados con árboles invertidos GIN (idx_synthetic_users_profiles) y soporte JSONB futuro.	Limitado en tipos de arreglo nativos: requiere tablas intermedias adicionales o parseo de cadenas JSON mediante OPENJSON.	Nativo para documentos y arreglos, pero sacrifica el esquema relacional estricto requerido por el catálogo de software y licencias.
3. Costo Operativo en Azure for Students	Muy predecible: el tier Burstable B1ms ofrece un costo mensual bajo y constante (~$12-15 USD/mes), ideal para los $100 USD de crédito anual estudiantil.	Complejo en niveles básicos: el modelo por DTU o vCore Serverless puede agotar créditos rápidamente si existen picos de conexión o	Tier gratuito limitado en conexiones y ancho de banda; fragmentaría la infraestructura fuera del grupo de recursos de Azure.

Criterio de Evaluación	Alternativa A (ELEGIDA): Azure Database for PostgreSQL Flexible Server 16	Alternativa B (Descartada): Azure SQL Database (MS SQL Server)	Alternativa C (Descartada): MongoDB Atlas / Azure Cosmos DB (NoSQL)
		almacenamiento.	
4. Costo / Desventaja Asumida por el Equipo	COSTO ASUMIDO: El nivel Flexible Server Burstable B1ms NO posee función de auto-pausa a cero (a diferencia de Azure SQL Serverless), por lo que consume créditos 24/7 mientras esté encendido, además de exigir configuración explícita de SSL (sslmode=require) y reglas de firewall IP. MITIGACIÓN: Monitoreo semanal de alertas de presupuesto en Azure Cost Management y apagado manual fuera de ventanas de desarrollo.	Dependencia de dialecto propietario T-SQL menos portable a entornos locales Docker/Linux livianos.	Pérdida de garantías relacionales y mayor riesgo de datos huérfanos en relaciones N:M.

En cuanto al almacenamiento de los instaladores de aplicaciones y paquetes de controladores, se seleccionó Azure Blob Storage (cuenta driwinstorage, configurada en la región Canada Central bajo redundancia local LRS y nivel de acceso Frecuente / Hot) frente a almacenar binarios como objetos BYTEA dentro de PostgreSQL o servirlos desde el disco local del servidor Express. El costo asumido de esta decisión es un cargo mensual de almacenamiento de objetos (~$0,020 USD por GB al mes, lo que representa apenas ~$0,21 USD mensuales para los 10,34 GB totales del catálogo) y la necesidad de sincronizar las URLs en la tabla driver_packages y recommended_apps, a cambio de liberar por completo al motor PostgreSQL y a la API Node.js de la transferencia de archivos de varios gigabytes (Microsoft Corporation, 2026b).

2.3. Inventario de Dependencias Verificadas y Rol Técnico en DriWin

Las Tablas 5 y 6 detallan la totalidad de las librerías, paquetes y motores que conforman la arquitectura del proyecto, indicando sus versiones exactas verificadas en el registro oficial de npm al 21 de octubre de 2026 y el problema técnico puntual que cada una resuelve dentro de DriWin.

Tabla 5. Stack de paquetes verificados para la API REST (Node.js + Express + TypeScript).

Paquete / Tecnología	Versión Verificada	Capa donde Opera	Rol Técnico	Problema Técnico que Resuelve en DriWin
Node.js (Runtime)	v24.21.0 (LTS >=20)	Servidor Cloud	Entorno de ejecución JS/TS asíncrono	Ejecuta el servidor HTTP con alta concurrencia en operaciones I/O hacia Azure PostgreSQL.
express	5.3.0	Capa 3 (Ruteo / Controladores)	Framework web HTTP minimalista	Gestiona el enrutamiento modular (/api/v1/*), middlewares en cadena y captura nativa de promesas rechazadas en controladores asíncronos.

Paquete / Tecnología	Versión Verificada	Capa donde Opera	Rol Técnico	Problema Técnico que Resuelve en DriWin
pg + @types/pg	8.23.1	Capa 5 (Acceso a Datos)	Driver oficial PostgreSQL para Node.js	Administra el pool de conexiones TLS (pg.Pool) hacia Azure Flexible Server y ejecuta consultas parametrizadas ($1..$n) inmunes a inyección SQL.
zod	4.6.7	Capa 3 (Middleware Validación)	Validación de esquemas y parseo estático	Valida en tiempo de ejecución los payloads entrantes (formato hexadecimal de ven_id/dev_id, enteros positivos, enums de riesgo) antes de tocar la capa de servicio, devolviendo HTTP 400 detallado.
helmet	8.3.0	Capa 3 (Seguridad HTTP)	Middleware de cabeceras de seguridad	Inyecta cabeceras HTTP protectoras (Strict-Transport-Security, X-Content-Type-Options, Content-Security-Policy) mitigando ataques XSS y sniffing.
cors + @types/cors	2.8.6 / 2.8.19	Capa 3 (Seguridad HTTP)	Control de acceso entre orígenes (CORS)	Restringe el consumo de la API exclusivamente a los orígenes autorizados en lista blanca (http://localhost:5173 y https://driwin.vercel.app).
express-rate-limit	8.7.1	Capa 3 (Protección de Abuso)	Limitador de tasa de peticiones por IP	Limita el tráfico a 60 peticiones por minuto por IP, protegiendo el endpoint de recomendación y el pool de conexiones de ataques de denegación de servicio.
dotenv	18.0.8	Configuración Transversal	Cargador de variables de entorno	Inyecta las credenciales desde el archivo local .env hacia process.env, manteniendo las claves fuera del código fuente (Criterio B4).
morgan	1.12.1	Capa 3 (Observabilidad)	Logger HTTP de peticiones entrantes	Registra verbo, ruta, código de estado HTTP y tiempo de respuesta en milisegundos durante la demostración en vivo.
typescript + tsx	6.0.3 / 4.23.1	Compilación y Desarrollo	Compilador de tipos estáticos y ejecutor TS	Garantiza coherencia de tipos en las 3 capas del servidor y permite recarga en caliente durante el desarrollo.

Tabla 6. Stack verificado de la Capa de Presentación y de la Capa de Negocio Local (Tauri/Rust).

Paquete / Tecnología	Versión Resuelta	Capa Arquitectónica	Licencia	Función en el Ecosistema DriWin
react + react-dom	19.2.8	Capa 1 (Presentación)	MIT	Renderizado declarativo del cuestionario de intención, modal UAC y dashboard de 5 pestañas con concurrencia mediada por scheduler 0.27.0.

Paquete / Tecnología	Versión Resuelta	Capa Arquitectónica	Licencia	Función en el Ecosistema DriWin
vite + @vitejs/plugin-react	8.2.2 / 6.1.1	Capa 1 (Build & Dev)	MIT	Empaquetador ultrarrápido basado en Rolldown 1.2.7 (escrito en Rust) y LightningCSS 1.33.0 para compilación ESM.
oxlint	1.81.0	Calidad de Código	MIT	Linter estático de alto rendimiento que valida reglas de hooks de React y tipado TypeScript en milisegundos.
@tauri-apps/cli + api	2.12.4	Capa 2 (Negocio Local)	MIT / Apache-2.0	Puente de comunicación segura invoke() entre el Front-End en React y los comandos nativos escritos en Rust.
Rust + PowerShell / WMI	Stable / Win11	Capa 2 (Negocio Local)	MIT / Nativo OS	Ejecución de Get-CimInstance Win32_PnPEntity, instalación silenciosa de paquetes con pnputil.exe y purga de bloatware con Remove-AppxPackage.

3. Decisión Arquitectónica: Monolito Modular vs. Distribuido

3.1. Postura Adoptada: Monolito de Metadatos con Offloading Binario

En cumplimiento del criterio A3 de la Unidad 2, el equipo adopta una postura arquitectónica explícita y fundamentada: el servidor Back-End de DriWin se construye como un MONOLITO MODULAR DE METADATOS, complementado con el desacoplamiento deliberado del tráfico binario pesado hacia un servicio externo de almacenamiento de objetos en la nube (patrón de Offloading sobre Azure Blob Storage) y una base de datos relacional administrada (Azure PostgreSQL Flexible Server).

Rechazamos dividir la lógica del servidor en una arquitectura de microservicios (por ejemplo, separar un microservicio de usuarios, otro de hardware, otro de aplicaciones y otro de bloatware) porque para el alcance funcional, volumen transaccional y tamaño del equipo de DriWin, una arquitectura de microservicios introduciría latencia de red interna, complejidad de transacciones distribuidas y sobrecarga operativa sin aportar ningún beneficio técnico real. La Tabla 7 detalla qué partes de la solución operan de forma monolítica y cuáles están desacopladas por razones físicas.

Tabla 7. Clasificación arquitectónica de componentes: Monolito modular vs. Servicios externos.

Componente del Sistema	Patrón Arquitectónico	Justificación de Ingeniería
API REST de Servidor (server-api/)	Monolito Modular en Capas	Un único proceso Node.js/Express despliega todos los módulos de dominio (hardware, drivers, apps, bloatware, perfiles y suscripciones). Permite transacciones ACID locales en PostgreSQL y trazabilidad directa sin saltos de red entre contenedores.
Cliente de Escritorio (app-inicio-windows/ + src-tauri/)	Monolito de Cliente Rico (SPA + Binario Nativo)	La interfaz React y la capa local Rust se compilan en un único ejecutable instalable para el usuario final, evitando que el cliente deba instalar dependencias externas.
Almacenamiento de Instaladores (driwinstorage)	Servicio Externo Desacoplado (Offloading)	Separado físicamente de la API por el peso de los archivos (10,34 GB totales). El cliente descarga los ejecutables directamente desde los contenedores apps/ y drivers/ de Azure Blob Storage.
Persistencia Relacional (driwin_catalog)	Servicio Administrado Externo (PaaS)	Delegado a Azure Database for PostgreSQL Flexible Server para garantizar respaldos automáticos, cifrado TLS y aislamiento del almacenamiento respecto del cómputo de la API.

3.2. Defensa por Carga Esperada, Tamaño del Equipo y Costo de Operación

Nuestra decisión de mantener un monolito modular para la API REST se sostiene sobre cuatro pilares cuantitativos propios del caso de negocio de DriWin:

Pilar 1: Carga esperada y efecto matemático del patrón de Offloading

En DriWin, un dispositivo cliente realiza únicamente dos tipos de interacciones con la nube: (a) consultas de catálogo y resolución de recomendaciones al iniciar el programa, y (b) descarga de binarios pesados (.exe y .msi) para su instalación silenciosa. Como demuestra la Tabla 8, si el servidor API entregara

directamente los binarios, apenas 10 usuarios simultáneos descargando el controlador gráfico AMD Adrenalin (540 MB) y el editor DaVinci Resolve (2.900 MB) generarían 34,4 GB de tráfico saliente (egress), saturando el ancho de banda, los descriptores de archivo y la memoria RAM del proceso Node.js.

Al aplicar el patrón de Offloading hacia Azure Blob Storage, la API REST solo responde con un documento JSON de aproximadamente 1 KB que contiene la propiedad download_url. De este modo, 10 usuarios concurrentes representan únicamente 10 KB de transferencia para nuestro monolito y 10 consultas SQL indexadas que se resuelven en menos de 10 milisegundos gracias a los índices B-Tree (idx_hardware_lookup e idx_driver_profile). En consecuencia, el monolito escala por cantidad de peticiones de texto ligero y jamás por megabytes transferidos.

Tabla 8. Análisis de volumen de tráfico de red y efecto del patrón de Offloading binario.

Operación del Caso de Uso	Tamaño de Datos por Operación	Componente que Absorbe la Carga	Impacto sobre la CPU y RAM de la API Monolítica
1. Envío de huella de hardware (VEN_ID / DEV_ID) y perfil	~0,3 KB (Payload JSON POST)	API Monolítica (Express)	Insignificante (< 1 ms de parseo y validación con Zod).
2. Cruce relacional en PostgreSQL y respuesta de recomendación	~1,2 KB (Response JSON)	API Monolítica + PostgreSQL	Mínimo (~5-8 ms usando índice compuesto idx_hardware_lookup).
3. Descarga de paquete de driver gráfico (ej. NVIDIA CUDA / AMD)	540 MB a 700 MB por archivo	Azure Blob Storage (contenedor drivers/)	CERO (0 bytes transitan por la API; descarga directa Cliente <-> Blob).
4. Descarga de lote de aplicaciones (ej. Perfil Creator completo)	Hasta 3.960 MB (3,87 GB)	Azure Blob Storage (contenedor apps/)	CERO (0 bytes transitan por la API; CDN/Blob absorbe el ancho de banda).
5. Instalación desatendida (/S, /qn) y purga de bloatware	0 bytes de red (Cómputo local)	CPU del PC del Usuario (Capa Tauri/Rust)	CERO (el cómputo pesado ocurre en la máquina del cliente).

Pilar 2: Tamaño y estructura del equipo de desarrollo (Ley de Conway)

El equipo está conformado exactamente por 3 integrantes (Vicente Acevedo, Javier Cáceres y Joaquín Castillo). Adoptar una arquitectura distribuida de microservicios obligaría a estos 3 desarrolladores a administrar múltiples repositorios o paquetes independientes, configurar un API Gateway, manejar contratos de red internos (gRPC/REST), sincronizar migraciones de bases de datos separadas y mantener múltiples pipelines de despliegue. Esto multiplicaría por cuatro la carga operativa accidental (toil) sin agregar valor funcional al usuario. Un monolito modular permite que los 3 integrantes trabajen sobre una única base de código fuertemente tipada, donde las fronteras entre módulos se imponen mediante carpetas y contratos de interfaces TypeScript en lugar de llamadas de red propensas a fallos.

Pilar 3: Costo de operación bajo presupuesto acotado (Azure for Students)

El proyecto opera dentro del límite estricto de $100 USD anuales de la suscripción educativa Azure for Students. Nuestra arquitectura monolítica requiere únicamente: (1) una instancia de cómputo para la API web, (2) una instancia Azure PostgreSQL Flexible Server en tier Burstable B1ms, y (3) una cuenta de

Azure Blob Storage LRS Hot (~$0,21 USD/mes por los 10,34 GB de catálogo). Una arquitectura de microservicios requeriría levantar un clúster de contenedores (AKS o múltiples Container Apps), un bus de mensajería y múltiples bases de datos o esquemas aislados, agotando los créditos semestrales en pocas semanas.

3.3. Complejidad de Despliegue y Umbrales Objetivos de Revisión

El flujo de despliegue del monolito modular es lineal y reproducible en menos de 10 minutos en cualquier equipo limpio: basta con compilar el proyecto TypeScript (tsc), conectar la variable de entorno DATABASE_URL hacia el servidor PostgreSQL y ejecutar el proceso Node.js. No obstante, como arquitectos de software reconocemos que ninguna decisión es estática. La Tabla 9 establece los cinco umbrales cuantitativos y de negocio bajo los cuales el equipo revisaría esta postura para extraer servicios distribuidos específicos.

Tabla 9. Umbrales cuantitativos para la revisión de la postura monolítica hacia microservicios.

Dimensión / Señal de Alerta	Umbral Objetivo de Disparo	Evolución Arquitectónica Planificada al Superar el Umbral
1. Tráfico concurrente en recomendación	> 500 peticiones/segundo sostenidas o latencia p95 > 300 ms	Escalamiento vertical inicial del pool PostgreSQL; posteriormente, extracción del módulo /recommendations a un servicio independiente con caché en memoria (Azure Cache for Redis).
2. Automatización de Fase 2 (Web Scraping / Ingesta)	Activación de bots recolectores de versiones o catálogo > 10.000 componentes	Extracción obligatoria de un Worker / Microservicio de Ingesta asíncrono y aislado (cola de tareas) para que los procesos de scraping o verificación de hashes SHA-256 no bloqueen el Event Loop de la API pública.
3. Pasarela de pagos y facturación real B2B/B2C	Procesamiento de pagos reales y emisión tributaria (SII / Webpay)	Separación del dominio comercial (subscriptions y facturación) en un microservicio aislado con políticas de seguridad financiera estrictas y cumplimiento normativo independiente.
4. Distribución geográfica internacional	Usuarios fuera de Sudamérica con latencia de descarga > 200 ms	Incorporación de Azure Front Door / CDN delante de los contenedores de Azure Blob Storage y réplicas de lectura para PostgreSQL.
5. Crecimiento de la organización de ingeniería	Equipo > 8 desarrolladores divididos en células de dominio autónomas	Partición del monolito modular siguiendo las fronteras ya definidas en los directorios services/ y repositories/ (Catálogo Técnico vs. Perfiles y Cuentas).

4. Diseño y Contrato de la Interfaz de Programación REST

4.1. Criterio de Versionado, Cabeceras y Envoltorios Estándar de Datos y Error

La interfaz de programación de aplicaciones (API) de DriWin ha sido diseñada siguiendo estrictamente los principios arquitectónicos REST orientados a recursos sobre el protocolo HTTP/1.1 y HTTP/2 (Fielding, 2000; Vara Mesa et al., 2015). El contrato completo sirve como acuerdo formal inmutable entre el servidor Back-End y el cliente de escritorio.

- **Criterio de Versionado Explícito en URI (/api/v1):** Todas las rutas públicas incorporan el prefijo de versión mayor en el segmento inicial de la ruta (por ejemplo, /api/v1/driver-packages). Se eligió el versionado en URI por sobre cabeceras personalizadas (Accept-Version) porque facilita la inspección directa en navegadores y clientes de prueba (Postman, Thunder Client, cURL), simplifica las reglas de enrutamiento y permite que, ante un cambio incompatible en el futuro Hito 3 o en producción, coexista una rama /api/v2 sin romper los clientes de escritorio ya instalados en máquinas de usuarios.
- **Formato, Codificación y Cabeceras Estándar:** Todo intercambio de cuerpo utiliza Content-Type: application/json; charset=utf-8. Las operaciones de escritura (POST, PUT, PATCH, DELETE) y las consultas de recomendación exigen autenticación mediante la cabecera X-API-Key (o Authorization: Bearer <token>), además de soportar la cabecera X-Request-Id para trazabilidad en los registros del servidor e Idempotency-Key en peticiones POST.
- **Envoltorio Único de Respuesta Exitosa y de Error:** Para evitar que el cliente Front-End deba adivinar la forma de cada respuesta, el servidor utiliza dos estructuras JSON canónicas: un envoltorio exitoso compuesto por los objetos data y meta (para paginación y tiempos de consulta), y un envoltorio estandarizado de error con código semántico, mensaje legible por humanos, arreglo de detalles por campo validado, marca de tiempo ISO-8601 UTC y ruta invocada.

```
// 1. Estructura Canónica de Respuesta Exitosa (HTTP 200 OK / 201 Created)
{
  "data": { ...objeto_recurso_o_arreglo_de_recursos... },
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 12,
    "totalPages": 1,
    "timestamp": "2026-10-30T14:00:00.000Z"
  }
}

// 2. Estructura Canónica de Respuesta de Error (HTTP 4xx / 5xx)
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "El cuerpo de la petición contiene campos inválidos.",
    "details": [
      { "field": "ven_id", "issue": "Debe ser un código hexadecimal de 4 caracteres (ej: 10DE)."
},
      { "field": "file_size_mb", "issue": "El tamaño del archivo debe ser un entero mayor que 0."
}
    ],
```

```
    "timestamp": "2026-10-30T14:03:12.184Z",
    "path": "/api/v1/driver-packages",
    "requestId": "req_01J8ZK5VQ2"
  }
}
```

4.2. Mapeo de Errores SQLSTATE de PostgreSQL hacia Códigos de Estado HTTP

Una falla común en servicios web deficientes es devolver HTTP 200 OK o HTTP 500 genérico ante errores de reglas de negocio o violaciones de integridad relacional. En DriWin, el middleware centralizado middlewares/errorHandler.ts intercepta tanto las excepciones de validación de esquemas Zod como los códigos nativos SQLSTATE emitidos por el motor PostgreSQL 16 (PostgreSQL Global Development Group, 2026), traduciéndolos de manera determinista al código de estado HTTP semánticamente correcto, tal como se especifica en la Tabla 10.

Tabla 10. Catálogo de códigos de estado HTTP y mapeo técnico desde SQLSTATE (PostgreSQL 16).

Código Estado HTTP	Código Interno (error.code)	Origen / SQLSTATE PostgreSQL	Condición de Disparo en el Dominio DriWin
200 OK	—	Ejecución exitosa	Lectura (GET) o actualización total/parcial (PUT/PATCH) completada satisfactoriamente.
201 Created	—	INSERT ... RETURNING	Creación exitosa (POST) de un nuevo componente, paquete de driver, app o regla; incluye cabecera Location.
204 No Content	—	DELETE exitoso (rowCount = 1)	Eliminación física exitosa de un recurso; respuesta sin cuerpo de mensaje.
400 Bad Request	VALIDATION_ERROR	Zod Schema / SQLSTATE 23514 (check_violation)	JSON malformado, ven_id no hexadecimal, file_size_mb <= 0 (ck_app_size) o risk_level fuera de BAJO|MEDIO|ALTO (ck_bloatware_risk).
401 Unauthorized	UNAUTHORIZED	Middleware authenticate.ts	Ausencia o invalidez de la cabecera de seguridad X-API-Key en rutas protegidas.
403 Forbidden	FORBIDDEN	Middleware authenticate.ts	Clave de cliente de solo lectura intentando ejecutar operaciones destructivas (POST/PUT/DELETE).
404 Not Found	NOT_FOUND / HARDWARE_NOT_FOUND	rowCount = 0 en SELECT por ID	Identificador primario inexistente (ej. driver_id = 999) o firma VEN_ID/DEV_ID aún no registrada en el catálogo.
409 Conflict	CONFLICT	SQLSTATE 23505 (unique_violation) o 23503 en DELETE	Intento de duplicar una firma única (uq_hardware_signature, uq_driver_recommendation, app_name, email) o borrar un perfil referenciado (ON DELETE RESTRICT).
422 Unprocessable Entity	FK_VIOLATION / NO_DRIVER_FOR_PROFILE	SQLSTATE 23503 (foreign_key_violation) en INSERT/UPDATE	Se referencia un component_id, profile_id o tier_id que no existe en su tabla padre, o el hardware existe pero no posee driver para ese perfil/SO.

Código Estado HTTP	Código Interno (error.code)	Origen / SQLSTATE PostgreSQL	Condición de Disparo en el Dominio DriWin
429 Too Many Requests	RATE_LIMITED	Middleware express-rate-limit	El cliente supera el umbral de 60 peticiones por minuto por dirección IP.
500 Internal Server Error	INTERNAL_ERROR	Excepción no controlada / 42P01	Error interno inesperado; el servidor registra la traza en consola pero jamás expone el stack trace al cliente.
503 Service Unavailable	DB_UNAVAILABLE	SQLSTATE 53300 / Timeout pg.Pool	El servidor Azure PostgreSQL Flexible Server no responde al chequeo SELECT 1 o se alcanzó el límite del pool.

4.3. Tabla Maestra de Recursos y Operaciones CRUD

El Hito 2 exige como mínimo funcional al menos dos recursos del dominio con las cuatro operaciones básicas de persistencia (Crear, Leer, Actualizar y Borrar). El diseño de DriWin supera este umbral exponiendo CINCO recursos principales con soporte CRUD completo (hardware-components, driver-packages, recommended-apps, bloatware-rules y user-profiles), tres recursos complementarios de gestión comercial y tablas pivote, dos endpoints especializados de negocio para el cruce de recomendaciones y una sonda de salud operativa. La Tabla 11 presenta el mapa maestro de la interfaz de programación.

Tabla 11. Tabla maestra de recursos expuestos por la API REST (/api/v1) y cobertura CRUD.

#	Ruta Base del Recurso	Tabla Asociada en PostgreSQL	GET (Leer)	POST (Crear)	PUT / PATCH (Actualizar)	DELETE (Borrar)	Rol en el Sistema
1	/api/v1/hardware-components	hardware_components	[SI]	[SI]	[SI] (PUT)	[SI]	CRUD Completo (Recurso Principal 1): Catálogo de firmas físicas VEN_ID / DEV_ID.
2	/api/v1/driver-packages	driver_packages	[SI]	[SI]	[SI] (PUT)	[SI]	CRUD Completo (Recurso Principal 2): Paquetes de drivers vinculados a Azure Blob Storage.
3	/api/v1/recommended-apps	recommended_apps	[SI]	[SI]	[SI] (PUT)	[SI]	CRUD Completo (Recurso Principal 3): Instaladores desatendidos de

#	Ruta Base del Recurso	Tabla Asociada en PostgreSQL	GET (Leer)	POST (Crear)	PUT / PATCH (Actualizar)	DELETE (Borrar)	Rol en el Sistema
							software en contenedor apps/.
4	/api/v1/bloatware-rules	bloatware_rules	[SI]	[SI]	[SI] (PUT)	[SI]	CRUD Completo (Recurso Principal 4): Firmas Appx y comandos PowerShell de detección/purga.
5	/api/v1/user-profiles	user_profiles	[SI]	[SI]	[SI] (PUT)	[SI]	CRUD Completo (Recurso Principal 5): Perfiles de intención (Gamer, Creator, Office, Developer).
6	/api/v1/profile-apps	profile_apps	[SI]	[SI]	—	[SI]	Gestión de relación N:M entre perfiles de usuario y aplicaciones recomendadas.
7	/api/v1/synthetic-users	synthetic_users	[SI]	[SI]	[SI] (PATCH)	[SI]	CRUD de usuarios sintéticos y telemetría de dispositivos activos por cuenta.
8	/api/v1/subscription-tiers	subscription_tiers	[SI]	[SI]	[SI] (PUT)	[SI]	CRUD de la matriz comercial de planes y límites de equipos en pesos chilenos (CLP).
9	/api/v1/recommendations/driver	JOIN: drivers + hardware + profiles	—	[SI] (Consulta)	—	—	Endpoint Nuclear de Negocio: Recibe VEN_ID/DEV_ID + Perfil y retorna el driver óptimo y su URL

#	Ruta Base del Recurso	Tabla Asociada en PostgreSQL	GET (Leer)	POST (Crear)	PUT / PATCH (Actualizar)	DELETE (Borrar)	Rol en el Sistema
							Blob.
10	/api/v1/recommendations/apps	JOIN: profile_apps + apps + profiles	[SI]	—	—	—	Endpoint de Unión de Perfiles: Retorna el catálogo consolidado sin duplicados (DISTINCT) para N perfiles.
11	/api/v1/health	Sonda activa (SELECT 1)	[SI]	—	—	—	Verificación de disponibilidad del servicio y latencia de conexión con Azure PostgreSQL.

4.4. Especificación Detallada de Contratos por Recurso y Consultas SQL

A continuación, se detallan los contratos técnicos completos para los recursos CRUD principales y los endpoints de recomendación que se ejecutarán durante la demostración en vivo de 7 minutos.

4.4.1. Recurso 1: /api/v1/hardware-components (CRUD Completo)

Representa los componentes físicos detectables mediante PCI/USB o HDAUDIO. La Tabla 12 resume sus operaciones, seguida de los cuerpos JSON de petición y respuesta y la consulta SQL parametrizada del repositorio.

Tabla 12. Contrato de endpoints para el Recurso 1: hardware-components.

Verbo	Ruta Exacta	Parámetros (Path / Query)	Éxito HTTP	Errores Posibles HTTP
GET	/api/v1/hardware-components	Query: ?ven_id=1002&dev_id=1638&category=GPU&page=1&limit=20	200 OK	400 Bad Request, 500 Internal Error
GET	/api/v1/hardware-components/:id	Path: id (entero, component_id)	200 OK	400 Bad Request, 404 Not Found
POST	/api/v1/hardware-components	Header: X-API-Key · Body JSON: ven_id, dev_id, subsys_id, commercial_name, category	201 Created	400 Validation, 401 Unauthorized, 409 Conflict (uq_hardware_signature)

Verbo	Ruta Exacta	Parámetros (Path / Query)	Éxito HTTP	Errores Posibles HTTP
PUT	/api/v1/hardware-components/:id	Path: id · Header: X-API-Key · Body JSON: commercial_name, category, subsys_id	200 OK	400 Validation, 401 Unauthorized, 404 Not Found, 409 Conflict
DELETE	/api/v1/hardware-components/:id	Path: id · Query: ?cascade=true · Header: X-API-Key	204 No Content	401 Unauthorized, 404 Not Found, 409 Conflict (si tiene drivers y falta cascade=true)

```
// POST /api/v1/hardware-components -> Request Body JSON
{
  "ven_id": "8086",
  "dev_id": "9A49",
  "subsys_id": "GENERIC",
  "commercial_name": "Intel Iris Xe Graphics (Tiger Lake-LP GT2)",
  "category": "GPU"
}

// POST /api/v1/hardware-components -> Response 201 Created (Location: /api/v1/hardware-
components/5)
{
  "data": {
    "component_id": 5,
    "ven_id": "8086",
    "dev_id": "9A49",
    "subsys_id": "GENERIC",
    "commercial_name": "Intel Iris Xe Graphics (Tiger Lake-LP GT2)",
    "category": "GPU",
    "created_at": "2026-10-30T15:10:00.120Z"
  }
}

-- SQL Parametrizado en server-api/src/repositories/hardware.repository.ts:
INSERT INTO hardware_components (ven_id, dev_id, subsys_id, commercial_name, category)
VALUES ($1, $2, COALESCE($3, 'GENERIC'), $4, $5)
RETURNING component_id, ven_id, dev_id, subsys_id, commercial_name, category, created_at;
```

4.4.2. Recurso 2: /api/v1/driver-packages (CRUD Completo)

Gestiona los paquetes de controladores optimizados por perfil de intención y entorno de sistema operativo, almacenando la ruta hacia el contenedor drivers/ de Azure Blob Storage y los modificadores de instalación silenciosa. La Tabla 13 define su contrato HTTP.

Tabla 13. Contrato de endpoints para el Recurso 2: driver-packages.

Verbo	Ruta Exacta	Parámetros (Path / Query)	Éxito HTTP	Errores Posibles HTTP
GET	/api/v1/driver-packages	Query: ?component_id=2&profile_id=1&os_environment=Win11_x64&whql=true	200 OK	400 Bad Request, 500 Internal Error
GET	/api/v1/driv	Path: id (entero, driver_id)	200	400 Bad Request, 404 Not

Verbo	Ruta Exacta	Parámetros (Path / Query)	Éxito HTTP	Errores Posibles HTTP
	er-packages/:id		OK	Found
POST	/api/v1/driver-packages	Header: X-API-Key · Body JSON completo del paquete de driver	201 Created	400 Validation, 401 Unauthorized, 409 Conflict (uq_driver_recommendation), 422 FK Violation
PUT	/api/v1/driver-packages/:id	Path: id · Header: X-API-Key · Body JSON: version_number, release_date, download_url, file_size_mb, silent_install_flags	200 OK	400 Validation, 401 Unauthorized, 404 Not Found, 409 Conflict, 422 FK Violation
DELETE	/api/v1/driver-packages/:id	Path: id · Header: X-API-Key	204 No Content	401 Unauthorized, 404 Not Found

```
// POST /api/v1/driver-packages -> Request Body JSON
{
  "component_id": 2,
  "profile_id": 1,
  "os_environment": "Win11_x64",
  "edition_name": "NVIDIA GeForce Game Ready Driver",
  "version_number": "561.09",
  "release_date": "2026-09-11",
  "download_url": "https://driwinstorage.blob.core.windows.net/drivers/nvidia-gameready-561.09-
win11-x64.exe",
  "file_size_mb": 685,
  "silent_install_flags": "-s -noreboot",
  "is_whql_certified": true
}

// PUT /api/v1/driver-packages/5 -> Request Body JSON (Actualización de versión y URL en Azure
Blob Storage)
{
  "version_number": "565.90",
  "release_date": "2026-10-22",
  "download_url": "https://driwinstorage.blob.core.windows.net/drivers/nvidia-gameready-565.90-
win11-x64.exe",
  "file_size_mb": 692,
  "silent_install_flags": "-s -noreboot"
}
```

4.4.3. Recursos 3, 4 y 5: Aplicaciones, Reglas de Bloatware y Perfiles

La Tabla 14 sintetiza los contratos CRUD de los tres recursos restantes que completan el catálogo operativo de DriWin, cuyos instaladores reales ya fueron cargados y sincronizados en el contenedor apps/ de Azure Blob Storage.

Tabla 14. Contrato de endpoints para los Recursos 3, 4 y 5 (apps, bloatware y perfiles).

Recurso y Ruta Base	Operaciones Soportadas	Campos Clave del Payload JSON	Reglas de Integridad y Negocio Asociadas
Recurso 3: /api/v1/recommended-apps y /api/v1/recommended-apps/:id	GET (lista y filtro ?profile_id=) GET por :id POST (201) PUT por :id (200) DELETE por :id (204)	app_name, category, version_number, download_url (en driwinstorage/apps/*), file_size_mb, silent_install_flags	Restricción UNIQUE en app_name (HTTP 409) y CHECK ck_app_size (file_size_mb > 0 -> HTTP 400). Al borrar una app se limpian sus vínculos en profile_apps por ON DELETE CASCADE.
Recurso 4: /api/v1/bloatware-rules y /api/v1/bloatware-rules/:id	GET (filtro ?risk_level=) GET por :id POST (201) PUT por :id (200) DELETE por :id (204)	app_display_name, package_signature, detection_powershell_cmd, removal_powershell_cmd, risk_level, description	Restricción UNIQUE uq_bloatware_signature (HTTP 409) y CHECK ck_bloatware_risk IN ('BAJO','MEDIO','ALTO'). El servicio omite reglas MEDIO/ALTO (como *XboxGamingOverlay*) si el perfil activo es Gamer.
Recurso 5: /api/v1/user-profiles y /api/v1/user-profiles/:id	GET (incluye conteo de apps/drivers) GET por :id POST (201) PUT por :id (200) DELETE por :id (204)	profile_name, optimization_focus	Restricción UNIQUE en profile_name (HTTP 409). La clave foránea desde driver_packages posee ON DELETE RESTRICT: impide borrar un perfil en uso devolviendo HTTP 409 Conflict.

4.4.4. Endpoints de Negocio: /api/v1/recommendations/driver y /api/v1/recommendations/apps

Estos dos endpoints materializan la propuesta de valor de DriWin: reciben la telemetría de hardware extraída por la terminal de Windows junto con el arreglo de perfiles de intención seleccionados por el usuario, y devuelven el paquete exacto de controlador y la unión matemática sin duplicados de las aplicaciones recomendadas.

```
// 1. POST /api/v1/recommendations/driver -> Request Body JSON
{
  "ven_id": "1002",
  "dev_id": "1638",
  "subsys_id": "GENERIC",
  "os_environment": "Win11_x64",
  "profiles": ["Gamer"]
}

// Response 200 OK -> Cruce exacto para AMD Radeon Vega en perfil Gamer
{
  "data": {
    "component": {
      "component_id": 1,
      "ven_id": "1002",
      "dev_id": "1638",
      "commercial_name": "AMD Radeon(TM) Graphics (Vega Integrated)",
      "category": "GPU"
    },
    "recommendation": {
      "driver_id": 1,
      "profile_name": "Gamer",
```

```
      "os_environment": "Win11_x64",
      "edition_name": "AMD Software: Adrenalin Edition",
      "version_number": "24.9.1",
      "release_date": "2026-09-15",
      "download_url": "https://driwinstorage.blob.core.windows.net/drivers/amd-adrenalin-24.9.1-
win11-64bit.exe",
      "file_size_mb": 540,
      "silent_install_flags": "-install",
      "is_whql_certified": true
    },
    "offloading": {
      "strategy": "DIRECT_AZURE_BLOB",
      "container": "drivers",
      "binary_traverses_api": false
    }
  },
  "meta": { "query_time_ms": 6 }
}

// 2. GET /api/v1/recommendations/apps?profiles=Gamer,Creator -> Consulta SQL subyacente:
SELECT DISTINCT a.app_id, a.app_name, a.category, a.version_number,
       a.download_url, a.file_size_mb, a.silent_install_flags
FROM user_profiles p
INNER JOIN profile_apps pa    ON pa.profile_id = p.profile_id
INNER JOIN recommended_apps a ON a.app_id      = pa.app_id
WHERE p.profile_name = ANY($1::text[])
ORDER BY a.category, a.app_name;
```

5. Separación de Responsabilidades y Arquitectura en Capas

5.1. Árbol de Directorios del Proyecto y Clasificación en Cinco Capas

Para cumplir con el nivel Destacado del criterio B3 (12 puntos), la separación en capas no puede limitarse a una declaración teórica: cada archivo del proyecto tiene asignada una responsabilidad única y excluyente dentro del árbol de directorios. La Figura 2 presenta la estructura de carpetas del repositorio DriWinProyect integrando el cliente React, la capa local Tauri/Rust, el servidor Express en capas y el directorio de persistencia SQL.

Figura 2. Árbol completo de directorios y separación modular en capas del proyecto DriWin.

```
DriWinProyect/
├── .gitignore                                  # Reglas de exclusión de secretos (.env*,
*.pem, *.key) y builds
├── LICENSE                                     # Licencia MIT (Copyright 2026 Equipo DriWin)
├── README.md                                   # Manual raíz del repositorio e instrucciones
de levantamiento
├── THIRD_PARTY_LICENSES.md                     # Auditoría de 70 paquetes npm y descargos de
responsabilidad
│
├── app-inicio-windows/                         # === CAPA 1: PRESENTACIÓN / INTERFAZ CLIENTE
(React 19 + TS) ===
│   ├── .env.example                            # Plantilla de variables públicas del cliente
(VITE_API_BASE_URL)
│   ├── package.json                            # Dependencias de UI y scripts Vite
│   └── src/
│       ├── main.tsx                            # Punto de montaje del árbol React en el DOM
│       ├── App.tsx                             # Vistas de cuestionario, modal UAC y 5
pestañas del Dashboard
│       ├── services/
│       │   └── api.ts                          # Cliente HTTP tipado (fetch hacia /api/v1/*)
│       └── styles/
│           └── theme.css                       # Tokens de diseño visual y paleta corporativa
DriWin
│
├── src-tauri/                                  # === CAPA 2: NEGOCIO LOCAL (Rust + Tauri
2.12.4 en PC Cliente) ===
│   ├── Cargo.toml                              # Manifiesto de crates Rust (tauri, serde,
serde_json, thiserror)
│   ├── tauri.conf.json                         # Permisos de ventana y capabilities de
ejecución local
│   └── src/
│       ├── main.rs                             # Registro de invoke_handler para comandos
locales
│       ├── powershell.rs                       # Wrapper seguro sobre std::process::Command
│       └── commands/
│           ├── hardware.rs                     # Comando detect_hardware() -> extrae VEN_ID y
DEV_ID vía WMI
│           ├── drivers.rs                      # Comando install_driver(url, flags) ->
descarga de Blob y ejecuta
│           └── bloatware.rs                    # Comando remove_bloatware(cmd) -> ejecuta
Remove-AppxPackage
│
├── server-api/                                 # === CAPAS 3, 4 Y 5 DEL SERVIDOR BACK-END
(Node.js + Express + TS) ===
│   ├── .env.example                            # Plantilla de variables del servidor
```

```
(DATABASE_URL, API_KEY)
│   ├── package.json                          # Manifiesto del servidor (express, pg, zod,
helmet, cors)
│   ├── tsconfig.json                         # Configuración estricta del compilador
TypeScript
│   └── src/
│       ├── server.ts                         # Arranque del servidor HTTP y cierre limpio
(graceful shutdown)
│       ├── app.ts                            # Pipeline de middlewares (helmet -> cors ->
morgan -> routes -> errors)
│       ├── config/
│       │   ├── env.ts                        # Validación de variables de entorno al
arranque mediante Zod
│       │   └── db.ts                         # Instancia única de pg.Pool conectada a Azure
PostgreSQL (SSL)
│       ├── middlewares/
│       │   ├── authenticate.ts               # Verificación de cabecera X-API-Key (HTTP 401
/ 403)
│       │   ├── validate.ts                   # Validación de req.body / req.query contra
esquemas Zod (HTTP 400)
│       │   └── errorHandler.ts               # Traducción de códigos SQLSTATE (23505, 23503)
a respuestas HTTP
│       ├── routes/                           # --- CAPA 3A: ENRUTAMIENTO HTTP ---
│       │   ├── index.ts                      # Montaje central bajo el prefijo /api/v1
│       │   ├── hardware.routes.ts            # Rutas CRUD para /hardware-components
│       │   ├── drivers.routes.ts             # Rutas CRUD para /driver-packages
│       │   ├── apps.routes.ts                # Rutas CRUD para /recommended-apps
│       │   ├── bloatware.routes.ts           # Rutas CRUD para /bloatware-rules
│       │   ├── profiles.routes.ts            # Rutas CRUD para /user-profiles
│       │   └── recommendations.routes.ts     # Rutas POST /driver y GET /apps
│       ├── controllers/                      # --- CAPA 3B: CONTROLADORES HTTP ---
│       │   ├── hardware.controller.ts        # Extrae DTO de req, invoca al servicio y emite
res.status().json()
│       │   ├── drivers.controller.ts         # Sin consultas SQL ni reglas de negocio
internas
│       │   ├── apps.controller.ts
│       │   ├── bloatware.controller.ts
│       │   ├── profiles.controller.ts
│       │   └── recommendations.controller.ts
│       ├── services/                         # --- CAPA 4: LÓGICA DE NEGOCIO DEL SERVIDOR --
-
│       │   ├── hardware.service.ts           # Normalización de cadenas VEN/DEV y control de
borrado en cascada
│       │   ├── driver.service.ts             # Reglas de priorización WHQL / Game Ready /
Studio por perfil
│       │   ├── app.service.ts                # Consolidación de aplicaciones y cálculo de
peso total en MB
│       │   ├── bloatware.service.ts          # Regla de exclusión de riesgo MEDIO/ALTO para
el perfil Gamer
│       │   └── recommendation.service.ts     # Orquestación del cruce hardware x perfil y
metadatos de Offloading
│       ├── repositories/                     # --- CAPA 5A: ACCESO A DATOS (REPOSITORIOS
SQL) ---
│       │   ├── hardware.repository.ts        # Consultas SQL parametrizadas ($1..$n) sobre
hardware_components
│       │   ├── driver.repository.ts          # Consultas SQL parametrizadas sobre
driver_packages
│       │   ├── app.repository.ts             # Consultas SQL parametrizadas sobre
recommended_apps y profile_apps
│       │   ├── bloatware.repository.ts       # Consultas SQL parametrizadas sobre
bloatware_rules
```

```
│       │   └── profile.repository.ts            # Consultas SQL parametrizadas sobre
user_profiles
│       └── domain/                              # Interfaces TypeScript compartidas (DTOs) y
clase ApiError
│
├── db/                                          # === CAPA 5B: ESQUEMA Y SEMILLA PERSISTENTE
===
│   └── init.sql                                 # Script DDL + DML idempotente (8 tablas, 4
índices, semilla y tests)
│
└── docs/                                        # Documentación técnica, diagramas y auditoría
normativa
```

5.2. Regla Unidireccional de Dependencia y Aislamiento de Responsabilidades

En la arquitectura del servidor server-api/, se impone una regla de dependencia estrictamente unidireccional: routes -> controllers -> services -> repositories -> config/db (PostgreSQL). Ninguna capa inferior puede importar ni conocer objetos de una capa superior. En particular: (1) en los archivos de controllers/ está estrictamente prohibido escribir sentencias SQL o reglas de filtrado de perfiles; (2) en los archivos de services/ está prohibido acceder a los objetos Request o Response de Express; y (3) los archivos de repositories/ son el único lugar del sistema autorizado para invocar pool.query(). La Tabla 15 sintetiza las responsabilidades y prohibiciones de cada capa.

Tabla 15. Clasificación de archivos del repositorio según las 5 capas de la arquitectura.

Capa Arquitectónica	Directorio y Archivos Clave	Responsabilidad Exclusiva	Qué tiene PROHIBIDO hacer esta capa
1. Presentación (Cliente)	app-inicio-windows/src/{App.tsx, main.tsx, services/api.ts, styles/theme.css}	Renderizar la interfaz React, capturar la intención del usuario en el cuestionario y consumir la API REST vía HTTP.	Conectar directamente a PostgreSQL, almacenar credenciales de base de datos o ejecutar comandos de consola sin pasar por Tauri.
2. Negocio Local (Cliente Nativo)	src-tauri/src/{main.rs, powershell.rs, commands/hardware.rs, drivers.rs, bloatware.rs}	Interactuar con el sistema operativo Windows (WMI, pnputil, AppxPackage) y descargar binarios desde Azure Blob Storage.	Contener el catálogo de recomendaciones cableado en código o tomar decisiones de negocio que correspondan al servidor central.
3. Ruteo y Controladores (Servidor)	server-api/src/routes/*.routes.ts server-api/src/controllers/*.controller.ts server-api/src/middlewares/*	Declarar endpoints HTTP, validar payloads entrantes con Zod, invocar al método del servicio y formatear el código HTTP de salida.	Escribir consultas SQL (SELECT/INSERT/UPDATE/DELETE) o decidir qué driver corresponde a un perfil.
4. Servicio / Negocio (Servidor)	server-api/src/services/{recommendation, driver, hardware, app, bloatware}.service.ts	Aplicar las reglas de negocio de DriWin: cruce de hardware x perfil, protección de Xbox Game Bar en modo Gamer y armado de metadatos de Offloading.	Manipular cabeceras HTTP (req/res) o concatenar consultas SQL directamente contra el driver pg.
5. Acceso a Datos y	server-	Ejecutar sentencias SQL 100%	Conocer códigos de estado HTTP

Capa Arquitectónica	Directorio y Archivos Clave	Responsabilidad Exclusiva	Qué tiene PROHIBIDO hacer esta capa
Persistencia	api/src/repositories/*.repository.ts server-api/src/config/db.ts db/init.sql	parametrizadas ($1..$n) contra Azure PostgreSQL Flexible Server y mapear filas a objetos de dominio.	(200, 404) o alterar reglas de negocio según la interfaz visual.

5.3. Recorrido Completo de una Petición de Punta a Punta (Trazabilidad)

Para demostrar que las capas operan de forma integrada y no son simples carpetas vacías, la Tabla 16 documenta el recorrido completo paso a paso de una petición real de recomendación de controlador para el usuario sintético Javier Rodríguez (perfil Gamer, tarjeta gráfica integrada AMD Radeon Vega con VEN_ID = 1002 y DEV_ID = 1638), desde que el cliente detecta el hardware hasta que recibe la URL del contenedor drivers/ de Azure Blob Storage.

Tabla 16. Trazabilidad paso a paso del recorrido de una petición desde la ruta hasta la base de datos.

Paso	Capa Activa	Archivo y Función Invocada	Transformación del Dato / Operación Técnica Ejecutada
1	Capa 2 (Negocio Local)	src-tauri/src/commands/hardware.rs -> detect_hardware()	Ejecuta Get-CimInstance Win32_VideoController en PowerShell, parsea la cadena PCI\VEN_1002&DEV_1638 y retorna el struct HardwareSnapshot { ven_id: '1002', dev_id: '1638', os_environment: 'Win11_x64' }.
2	Capa 1 (Presentación)	app-inicio-windows/src/services/api.ts -> postDriverRecommendation(dto)	Envía petición HTTP POST a /api/v1/recommendations/driver con cabecera X-API-Key y cuerpo JSON: {'ven_id':'1002','dev_id':'1638','os_environment':'Win11_x64','profiles':['Gamer']}.
3	Capa 3 (Middlewares)	server-api/src/app.ts -> helmet() -> cors() -> authenticate.ts -> validate.ts	Verifica origen permitido en CORS, valida la clave X-API-Key y comprueba mediante el esquema Zod DriverRecommendationSchema que ven_id y dev_id sean hexadecimales de 4 dígitos.
4	Capa 3A (Ruteo)	server-api/src/routes/recommendations.routes.ts -> router.post('/driver', ...)	Dirige la petición validada hacia el método controlador correspondiente.
5	Capa 3B (Controlador)	server-api/src/controllers/recommendations.controller.ts -> getDriverRecommendation(req, res, next)	Extrae el DTO limpio desde req.body e invoca de forma asíncrona a await recommendationService.resolveOptimalDriver(dto).
6	Capa 4 (Servicio)	server-api/src/services/recommendation.service.ts -> resolveOptimalDriver(dto)	Normaliza los códigos VEN/DEV a mayúsculas, verifica la existencia del componente mediante hardwareRepository.findBySignature('1002','1638') y solicita los paquetes candidatos a driverRepository.findByHardwareAndProfiles().
7	Capa 5A (Repositorio)	server-api/src/repositories/driver.repository.ts -> findByHardwareAndProfiles('1002','1638','Win	Prepara la sentencia SQL parametrizada con INNER JOIN entre driver_packages, hardware_components y user_profiles usando los marcadores $1, $2, $3 y $4, e

Paso	Capa Activa	Archivo y Función Invocada	Transformación del Dato / Operación Técnica Ejecutada
		11_x64',['Gamer'])	invoca pool.query(sql, params).
8	Capa 5B (Base de Datos)	Azure PostgreSQL Flexible Server (driwin_catalog) -> Esquema public	El planificador de PostgreSQL utiliza el índice compuesto idx_hardware_lookup (ven_id, dev_id) y el índice idx_driver_profile (profile_id), localizando la fila driver_id = 1 en ~6 ms.
9	Capa 4 (Servicio)	server-api/src/services/recommendation.service.ts	Recibe la fila desde el repositorio, aplica la política de negocio (selecciona AMD Software: Adrenalin Edition v24.9.1 con flag '-install') y adjunta el descriptor de Offloading hacia el contenedor drivers/ de driwinstorage.
10	Capa 3B (Controlador)	server-api/src/controllers/recommendations.controller.ts	Envuelve el resultado en la estructura canónica { data, meta } y emite res.status(200).json(...).
11	Capa 1 y 2 (Cliente + Blob)	App.tsx -> installDriver() -> src-tauri/src/commands/drivers.rs	La interfaz muestra el driver recomendado; al confirmar el usuario, la capa local descarga el binario de 540 MB directamente desde https://driwinstorage.blob.core.windows.net/drivers/amd-adrenalin-24.9.1-win11-64bit.exe y lo instala con el flag '-install'.

6. Persistencia en PostgreSQL, Modelo de Datos y Seguridad

6.1. Diseño del Script Reproducible e Idempotente (db/init.sql)

La persistencia del sistema reside en un motor real de base de datos relacional (PostgreSQL 16 en Azure Database for PostgreSQL Flexible Server, base de datos driwin_catalog) y jamás en memoria volátil ni en archivos temporales. El archivo db/init.sql (449 líneas) ubicado en el repositorio ha sido diseñado bajo el principio de idempotencia estricta: inicia ejecutando sentencias DROP TABLE IF EXISTS ... CASCADE en orden inverso de dependencia (tablas hijas hacia tablas padres: synthetic_users, profile_apps y driver_packages primero; subscription_tiers, bloatware_rules, recommended_apps, hardware_components y user_profiles después), seguido de la creación de las 8 tablas (DDL), los 4 índices de rendimiento, la inserción del catálogo semilla completo (DML) y 9 consultas analíticas de verificación.

6.2. Diccionario de Datos, Claves, Restricciones e Índices de las 8 Tablas

La Figura 3 representa el Diagrama Entidad-Relación (DER) del esquema public, y la Tabla 17 detalla el diccionario relacional de las 8 tablas con sus claves primarias, claves foráneas, políticas de borrado referencial, restricciones de dominio (CHECK / UNIQUE) e índices.

Figura 3. Diagrama Entidad-Relación (DER) textual de las 8 tablas en PostgreSQL 16.

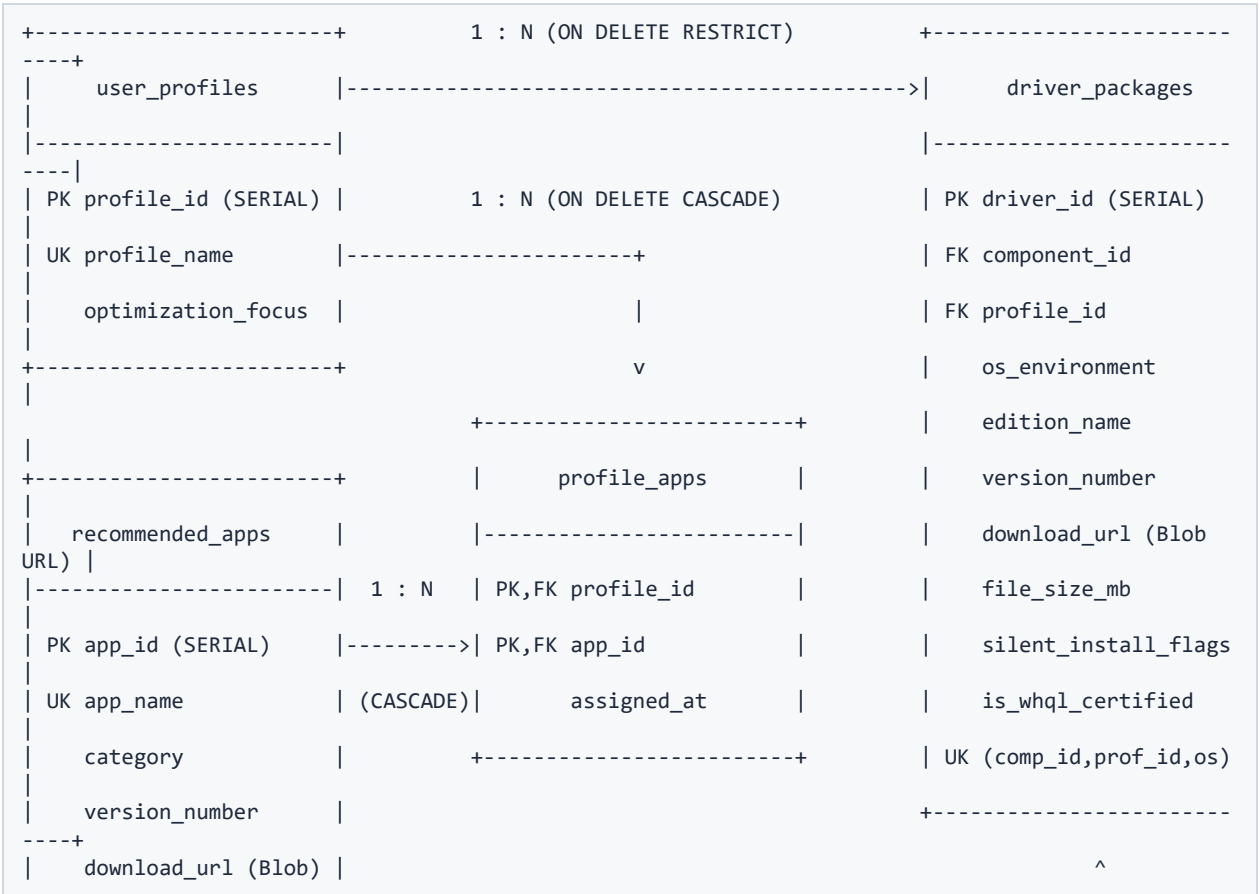




Tabla 17. Diccionario relacional consolidado de las 8 tablas del esquema driwin_catalog.

Tabla	Clave Primaria (PK)	Claves Foráneas (FK) y Política	Restricciones UNIQUE / CHECK e Índices	Filas Semilla y Propósito en el Dominio
1. user_profiles	profile_id (SERIAL)	Ninguna (Tabla Padre Raíz)	UNIQUE (profile_name)	4 filas: Gamer (1), Creator (2), Office (3), Developer (4). Define el eje de intención del usuario.
2. hardware_components	component_id (SERIAL)	Ninguna (Tabla Padre de Hardware)	uq_hardware_signature UNIQUE (ven_id, dev_id, subsys_id)	4 filas base: AMD Vega (1002:1638), NVIDIA RTX 3060 (10DE:2504), MediaTek

Tabla	Clave Primaria (PK)	Claves Foráneas (FK) y Política	Restricciones UNIQUE / CHECK e Índices	Filas Semilla y Propósito en el Dominio
			Índice B-Tree: idx_hardware_lookup (ven_id, dev_id)	MT7921 (14C3:7961), Realtek Audio (10EC:0256).
3. driver_packages	driver_id (SERIAL)	fk_driver_component -> hardware_components (ON DELETE CASCADE) fk_driver_profile -> user_profiles (ON DELETE RESTRICT)	uq_driver_recommendation UNIQUE (component_id, profile_id, os_environment) Índice B-Tree: idx_driver_profile (profile_id)	12 filas: Mapea cada componente y perfil a su instalador exacto en el contenedor drivers/ de Azure Blob Storage.
4. recommended_apps	app_id (SERIAL)	Ninguna (Tabla Padre de Software)	UNIQUE (app_name) CHECK ck_app_size (file_size_mb > 0)	20 filas (5 por perfil): Instaladores verificados en el contenedor apps/ (Steam, Blender, DaVinci Resolve, VS Code, Docker, LibreOffice, 7-Zip, etc.).
5. profile_apps	PK Compuesta: (profile_id, app_id)	fk_pa_profile -> user_profiles (ON DELETE CASCADE) fk_pa_app -> recommended_apps (ON DELETE CASCADE)	PK Compuesta evita duplicados Índice B-Tree: idx_profile_apps_app (app_id)	20 vínculos N:M: Permite resolver la unión de aplicaciones cuando un usuario elige perfiles híbridos (ej. Gamer + Creator).
6. bloatware_rules	bloatware_id (SERIAL)	Ninguna (Catálogo Universal de Reglas)	uq_bloatware_signature UNIQUE (package_signature) CHECK ck_bloatware_risk (risk_level IN ('BAJO','MEDIO','ALTO'))	3 reglas semilla (*BingNews*, *XboxGamingOverlay*, *MicrosoftSolitaireCollection*) con scripts PowerShell de detección y remoción.
7. subscription_tiers	tier_id (SERIAL)	Ninguna (Tabla Padre Comercial)	UNIQUE (tier_name) CHECK ck_tier_price (price_clp >= 0) CHECK ck_tier_devices (max_devices > 0)	5 planes comerciales en CLP: Pase Mensual ($5.000), Compra Vitalicia ($30.000), Builder Pro, Pack Volumen ($1.500.000) y Alianza OEM.
8. synthetic_users	user_id (SERIAL)	fk_user_tier -> subscription_tiers (ON DELETE RESTRICT)	UNIQUE (email) CHECK ck_profiles_not_empty (cardinality >= 1) CHECK ck_active_devices (>= 0) Índice GIN: idx_synthetic_users_profiles	6 usuarios sintéticos (Javier Rodríguez, Camila Rojas, Diego Salazar, Valentina Muñoz, Rodrigo Contreras, IEC Los Alerces).

6.3. Manejo de Credenciales, Variables de Entorno (.env.example) y Seguridad

En cumplimiento estricto del criterio B4 y de las reglas duras del curso, el repositorio fue sometido a una auditoría exhaustiva de seguridad sobre la totalidad de sus 41 confirmaciones históricas en Git. El barrido demostró CERO credenciales, contraseñas o cadenas de conexión expuestas en el árbol de trabajo y en toda la historia versionada. El archivo .gitignore de la raíz (y su homólogo en app-inicio-windows/.gitignore como defensa en profundidad) bloquea cualquier archivo .env, .env.*, *.pem, *.key y *.pfx, permitiendo explícitamente (!.env.example) versionar la plantilla de configuración de ejemplo.

```
# ==============================================================================
# Archivo: server-api/.env.example (Plantilla sin datos reales para el repositorio)
# Instrucción: Copiar a server-api/.env y completar con credenciales propias
# ==============================================================================

# 1. Configuración del Servidor HTTP
NODE_ENV=development
PORT=3000
API_BASE_PATH=/api/v1
ALLOWED_ORIGINS=http://localhost:5173,https://driwin.vercel.app
API_KEY=REEMPLAZAR_CON_CLAVE_HEXADECIMAL_DE_32_CARACTERES
RATE_LIMIT_WINDOW_MS=60000
RATE_LIMIT_MAX=60

# 2. Conexión a Azure Database for PostgreSQL Flexible Server (TLS Obligatorio)
DATABASE_URL=postgresql://usuario_db:REEMPLAZAR_PASSWORD@driwin-db-
server.postgres.database.azure.com:5432/driwin_catalog?sslmode=require
PGSSLMODE=require
DB_POOL_MAX=10
DB_STATEMENT_TIMEOUT_MS=5000

# 3. Configuración de Azure Blob Storage (Offloading de instaladores y drivers)
AZURE_STORAGE_ACCOUNT_NAME=driwinstorage
AZURE_BLOB_ENDPOINT=https://driwinstorage.blob.core.windows.net
AZURE_CONTAINER_APPS=apps
AZURE_CONTAINER_DRIVERS=drivers
AZURE_SAS_TOKEN=
```

Tabla 18. Auditoría de seguridad, barrido de secretos en Git y matriz de controles técnicos.

Vector de Riesgo	Evidencia Auditada en el Repositorio	Control Técnico Implementado / Diseñado
1. Filtración de contraseñas en Git	Ejecución de git log --all -S'password' y git grep sobre los 41 commits: 0 coincidencias.	Reglas .env, .env.*, *.pem, *.key en .gitignore (commit e71d85d) y validación al arranque en config/env.ts.
2. Inyección SQL (SQLi)	Consultas en db/init.sql y capa de repositorios.	Uso exclusivo de consultas parametrizadas ($1, $2..$n) mediante pg.Pool; cero concatenación de strings.
3. Tráfico en texto plano hacia la BD	Conexión hacia Azure PostgreSQL Flexible Server en Canada Central.	Parámetro obligatorio sslmode=require (TLS 1.2/1.3) y lista blanca de IPs en el firewall de Azure.
4. Ejecución arbitraria de comandos PowerShell	Columna removal_powershell_cmd en bloatware_rules.	El cliente Rust jamás recibe comandos arbitrarios desde el usuario; solo ejecuta firmas preaprobadas en el catálogo protegido por X-API-Key.

7. Instrucciones de Ejecución Reproducible y Demostración en Vivo

Para satisfacer la puerta de entrada del Hito 2 (que exige que el proyecto pueda levantarse en otro equipo siguiendo las instrucciones entregadas) y respaldar el criterio A1 (Demostración de la API en vivo ejecutando al menos cuatro peticiones reales que cubran crear, leer, actualizar y borrar mostrando el código de estado y el dato persistido), a continuación se detalla la secuencia exacta de comandos en terminal.

7.1. Paso a Paso para Levantar la Base de Datos y el Servidor en un Equipo Limpio

```
# 1. Clonar el repositorio oficial y posicionarse en la rama del proyecto
git clone https://github.com/VinJimbo97/DriWinProyect.git
cd DriWinProyect

# 2. Crear la base de datos driwin_catalog (una única vez en PostgreSQL local o Azure Flexible
Server)
psql "host=driwin-db-server.postgres.database.azure.com port=5432 user=driwin_admin
dbname=postgres sslmode=require" -c "CREATE DATABASE driwin_catalog;"

# 3. Ejecutar el script reproducible e idempotente de esquema y semilla (8 tablas + 32 URLs de
Azure Blob Storage)
psql "host=driwin-db-server.postgres.database.azure.com port=5432 user=driwin_admin
dbname=driwin_catalog sslmode=require" -v ON_ERROR_STOP=1 -f db/init.sql

# 4. Configurar las variables de entorno del servidor API
cd server-api
cp .env.example .env
# (Editar .env con la cadena DATABASE_URL real y definir API_KEY=<API_KEY>)

# 5. Instalar dependencias exactamente desde el lockfile y levantar el servidor Express en modo
desarrollo
npm ci
npm run dev
# El servidor queda escuchando en http://localhost:3000/api/v1/health
```

7.2. Secuencia de 4 Peticiones Reales (CRUD Completo) para la Demostración en Vivo

La colección exportada en el repositorio (y lista para ejecutarse con cURL, Postman o Thunder Client) ejecuta en orden las cuatro operaciones del ciclo de vida del recurso /api/v1/driver-packages y /api/v1/hardware-components:

```
# PETICIÓN 1 (CREAR - POST -> Devuelve HTTP 201 Created y persiste en PostgreSQL)
curl -i -X POST http://localhost:3000/api/v1/hardware-components   -H "Content-Type:
application/json"   -H "X-API-Key: <API_KEY>"   -d
'{"ven_id":"8086","dev_id":"9A49","subsys_id":"GENERIC","commercial_name":"Intel Iris Xe
Graphics","category":"GPU"}'

# PETICIÓN 2 (LEER - GET -> Devuelve HTTP 200 OK verificando que el registro component_id=5 quedó
guardado en la BD)
curl -i -X GET "http://localhost:3000/api/v1/hardware-components?ven_id=8086&dev_id=9A49"   -H
"Accept: application/json"

# PETICIÓN 3 (ACTUALIZAR - PUT -> Devuelve HTTP 200 OK actualizando el nombre comercial en
```

```
PostgreSQL)
curl -i -X PUT http://localhost:3000/api/v1/hardware-components/5   -H "Content-Type:
application/json"   -H "X-API-Key: <API_KEY>"   -d '{"commercial_name":"Intel Iris Xe Graphics G7
(WHQL Validated)","category":"GPU"}'

# PETICIÓN 4 (BORRAR - DELETE -> Devuelve HTTP 204 No Content eliminando el registro de prueba)
curl -i -X DELETE "http://localhost:3000/api/v1/hardware-components/5?cascade=true"   -H "X-API-
Key: <API_KEY>"

# PETICIÓN ADICIONAL DE NEGOCIO (Recomendación de Driver con Offloading hacia Azure Blob Storage
-> HTTP 200 OK)
curl -i -X POST http://localhost:3000/api/v1/recommendations/driver   -H "Content-Type:
application/json"   -H "X-API-Key: <API_KEY>"   -d
'{"ven_id":"1002","dev_id":"1638","os_environment":"Win11_x64","profiles":["Gamer"]}'
```

Conclusión

A lo largo del desarrollo del Hito 2, el equipo logró transformar la promesa conceptual formulada en septiembre en una arquitectura de servidor sólida, verificable y alineada con las exigencias reales de un producto de software en la nube. Se consolidó y desplegó un modelo de datos relacional normalizado de 8 tablas en Azure Database for PostgreSQL Flexible Server 16 mediante un script de creación completamente reproducible e idempotente (db/init.sql), se aprovisionó la infraestructura de almacenamiento de objetos en Azure Blob Storage (driwinstorage) poblando los contenedores apps/ y drivers/ con instaladores reales parametrizados para ejecución silenciosa, y se estructuró el contrato completo de la API REST (/api/v1) bajo una separación estricta en capas (enrutamiento, controladores, servicios de negocio y repositorios SQL).

Desde la perspectiva de toma de decisiones de ingeniería, el aprendizaje más valioso del equipo consistió en comprender que una arquitectura eficiente no es aquella que acumula tecnologías complejas por moda, sino la que asigna cada herramienta a la naturaleza física de su carga de trabajo. La decisión de adoptar un enfoque dual —manteniendo Rust (Tauri) confinado a la ejecución local privilegiada en la máquina del cliente y adoptando Node.js con Express y TypeScript para la API REST en la nube—, sumada al patrón de Offloading binario hacia Azure Blob Storage, nos permitió resolver simultáneamente el cuello de botella de transferencia de archivos de varios gigabytes, proteger el presupuesto de créditos estudiantiles y unificar el tipado de datos de extremo a extremo.

Finalmente, declaramos con total transparencia las tareas que quedan comprometidas para la integración del Hito 3: (1) sustituir los arreglos simulados (mocks en useState) del componente App.tsx en el Front-End por llamadas reales al servicio mediante el cliente HTTP tipado; (2) homologar en la interfaz de usuario las denominaciones canónicas de los perfiles (Gamer, Creator, Developer, Office) y los precios en pesos chilenos (CLP) provenientes de la tabla subscription_tiers; y (3) empaquetar los comandos locales de Tauri para cerrar el ciclo de punta a punta desde el clic del usuario hasta la instalación silenciosa en Windows.

Bibliografía

A continuación, se listan en estricto orden alfabético y bajo norma APA séptima edición las referencias de documentación técnica oficial, textos metodológicos de ingeniería de software y cuerpos normativos citados en el desarrollo del presente documento:

Acevedo, V., Cáceres, J. y Castillo, J. (2026). Propuesta técnica Hito 1: Proyecto DriWin. Escuela de Ingeniería Civil en Computación e Informática, Universidad Mayor.

Cámara Nacional de Comercio, Servicios y Turismo de Chile [CNC]. (2025). Índice de adaptación tecnológica del comercio minorista, hoteles y restaurantes: Segundo semestre de 2025. Departamento de Estudios CNC. https://cnc.cl/

Cardador Cabello, A. L. (2015). Desarrollo de aplicaciones web distribuidas (UF1846). IC Editorial.

Colinhacks. (2026). Zod: TypeScript-first schema validation with static type inference (Versión 4.6) [Documentación oficial]. https://zod.dev/

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Granados La Paz, R. L. (2015). Desarrollo de aplicaciones web en el entorno servidor (UF1844). IC Editorial.

Ley N.º 20.920. Establece marco para la gestión de residuos, la responsabilidad extendida del productor y fomento al reciclaje. (1 de junio de 2016). Diario Oficial de la República de Chile. https://www.bcn.cl/leychile/navegar?idNorma=1090894

Ley N.º 21.663. Ley marco de ciberseguridad. (8 de abril de 2024). Diario Oficial de la República de Chile. https://www.bcn.cl/leychile/navegar?idNorma=1202434

Ley N.º 21.719. Regula la protección y el tratamiento de los datos personales y crea la Agencia de Protección de Datos Personales. (13 de diciembre de 2024). Diario Oficial de la República de Chile. https://www.bcn.cl/leychile/navegar?idNorma=1209272

Meta Platforms, Inc. (2026). React 19 official documentation [Documentación técnica]. https://react.dev/

Microsoft Corporation. (2026a). Azure Database for PostgreSQL - Flexible Server documentation [Documentación técnica oficial]. Microsoft Learn. https://learn.microsoft.com/en-us/azure/postgresql/flexible-server/

Microsoft Corporation. (2026b). Azure Blob Storage documentation: Block blobs, access tiers, and Shared Access Signatures (SAS) [Documentación técnica oficial]. Microsoft Learn. https://learn.microsoft.com/en-us/azure/storage/blobs/

Microsoft Corporation. (2026c). TypeScript language specification and handbook (Versión 6.0) [Documentación técnica oficial]. https://www.typescriptlang.org/docs/

Microsoft Corporation. (2026d). PnPUtil command syntax and Windows Management Instrumentation (WMI/CIM) provider reference [Documentación técnica oficial]. Microsoft Learn. https://learn.microsoft.com/en-us/windows-hardware/drivers/devtest/pnputil

Ministerio del Medio Ambiente. (2026). Estrategia nacional de gestión de residuos de aparatos eléctricos y electrónicos (RAEE) en Chile. Gobierno de Chile. https://mma.gob.cl/

OpenJS Foundation. (2026a). Express 5.x API reference [Documentación técnica oficial]. https://expressjs.com/en/5x/api.html

OpenJS Foundation. (2026b). Node.js v24.x LTS documentation: Event loop, timers, and process management [Documentación técnica oficial]. https://nodejs.org/docs/latest/api/

PostgreSQL Global Development Group. (2026). PostgreSQL 16.0 documentation: Appendix A. PostgreSQL Error Codes (SQLSTATE) & GIN Indexes [Documentación técnica oficial]. https://www.postgresql.org/docs/16/index.html

Pressman, R. S. (2009). Ingeniería de software: Un enfoque práctico (7.ª ed.). McGraw-Hill.

Tauri Contributors. (2026). Tauri 2.0 documentation: Inter-process communication, capabilities, and security model [Documentación técnica oficial]. https://v2.tauri.app/

Vara Mesa, J. M., Verde Marín, J. y López Sanz, M. (2015). Desarrollo web en entorno servidor. RA-MA Editorial.

Anexos

Anexo 1: Declaración de Uso de Inteligencia Artificial (Regla 7 y Anexo C)

En cumplimiento de las normas de integridad académica de la Escuela de Ingeniería Civil de la Universidad Mayor (Anexo C del programa INGT1003), el equipo declara de forma transparente el uso de herramientas de Inteligencia Artificial durante el desarrollo del Hito 2:

- **1. Herramientas utilizadas:** Google Gemini y asistente de código integrado en Visual Studio Code (Continue / Copilot).
- **2. Partes del trabajo en las que se emplearon:** (a) Auditoría estática del árbol de directorios del repositorio e inspección del historial de confirmaciones en Git para verificar la ausencia de credenciales expuestas; (b) asistencia en la redacción de las consultas SQL de prueba con índices GIN y uniones de arreglos (unnest) en db/init.sql; (c) orientación en la configuración de contenedores de Azure Blob Storage (bloques de 16 MiB y desactivación de blobs en páginas .vhd); y (d) estructuración editorial y tablas de contrato HTTP del presente informe técnico.
- **3. Método de verificación de los resultados:** Ningún fragmento de código SQL, configuración de infraestructura ni especificación de contrato fue incorporado a ciegas. El script db/init.sql fue ejecutado y verificado directamente sobre nuestra instancia real de Azure Database for PostgreSQL Flexible Server desde el cliente de base de datos en VS Code, comprobando los conteos de filas, las restricciones de integridad referencial y los planes de ejecución. Asimismo, los 19 instaladores iniciales y los paquetes de controladores fueron cargados y verificados en nuestra cuenta driwinstorage del Portal de Azure. Todos los integrantes del equipo (Vicente Acevedo, Javier Cáceres y Joaquín Castillo) comprenden íntegramente el diseño en capas, el modelo relacional y el contrato REST, estando preparados para explicar y defender cualquier sección en el sorteo individual.

Anexo 2: Análisis de Protección de Datos Personales — Ley N.º 21.719 (Anexo D)

Considerando la entrada en vigencia plena de la Ley N.º 21.719 sobre Protección de Datos Personales en Chile el 1 de diciembre de 2026, el diseño del Back-End de DriWin incorpora el principio de privacidad desde el diseño y por defecto (Privacy by Design):

- **Minimización de datos en el escaneo de hardware:** El endpoint POST /api/v1/recommendations/driver recibe exclusivamente códigos técnicos anónimos de fabricante y modelo de chip (ven_id, dev_id, subsys_id y os_environment). El sistema NO recolecta números de serie de placa madre, direcciones MAC, direcciones IP persistentes, nombres de usuario de Windows ni archivos personales.
- **Uso exclusivo de datos sintéticos en entornos de evaluación:** Los 6 registros contenidos en la tabla synthetic_users corresponden a identidades ficticias creadas exclusivamente para validar las restricciones de planes comerciales (subscription_tiers). No se ha cargado ningún dato personal real al repositorio ni al servidor de Azure.
- **Medidas técnicas de seguridad y derecho de supresión:** Las conexiones a la base de datos exigen cifrado en tránsito (sslmode=require), el almacenamiento en Azure Blob Storage cuenta con cifrado

en reposo AES-256 mediante claves administradas por Microsoft (MMK), y el endpoint DELETE /api/v1/synthetic-users/:id garantiza el borrado físico inmediato ante cualquier solicitud de eliminación.

Anexo 3: Matriz de Deuda de Integración UI-BD y Checklist de Cierre hacia el Hito 3

La auditoría cruzada entre el prototipo Front-End (app-inicio-windows/src/App.tsx) y el modelo relacional del Back-End (db/init.sql) permitió identificar 6 puntos de divergencia que se documentan como hoja de ruta para la integración del Hito 3:

ID	Elemento Auditado	Estado en Prototipo UI (App.tsx)	Estado en Base de Datos (db/init.sql)	Acción de Integración Programada
A-1	Nombres de Perfiles	Gamer, Desarrollador, Creador, Oficinista	Gamer, Developer, Creator, Office	Homologar el diccionario de perfiles en la UI a los nombres canónicos en inglés o mapear en profile.service.ts.
A-2	Lista de Drivers	Arreglo estático en useState (líneas 39-46)	12 paquetes reales en tabla driver_packages	Reemplazar useState estático por llamada a POST /api/v1/recommendations/driver.
A-3	Catálogo de Apps	4 aplicaciones fijas de ejemplo en la vista	20 aplicaciones en recommended_apps con URLs de Azure Blob	Consumir GET /api/v1/recommendations/apps?profiles=... al montar la pestaña Apps.
A-4	Reglas de Bloatware	4 ítems mock (OneDrive, Cortana, Clipchamp, Xbox)	Tabla bloatware_rules con comandos PowerShell y risk_level	Consumir GET /api/v1/bloatware-rules aplicando el filtro de seguridad para el perfil Gamer.
A-5	Moneda y Planes Pro	Mostrado en dólares ($9.99 USD anual)	Tabla subscription_tiers expresada en pesos chilenos ($5.000 / $30.000 CLP)	Renderizar dinámicamente la pestaña Premium leyendo GET /api/v1/subscription-tiers.
A-6	Rama Principal Git	Avances del Hito 2 en rama feature/auditoria-vibecode	Rama main 4 commits detrás de la rama de trabajo	Ejecutar Pull Request y merge hacia main antes del cierre de entrega del Hito 2.